ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ
Факултет Компютърни системи и технологии
Катедра „Компютърни системи“

ДИПЛОМНА РАБОТА

за придобиване на образователно-квалификационна степен
„бакалавър“

на тема

„Платформа за публикуване и слушане на музика“

Дипломант:
Димитър Тодоров Колев
Фак. номер: 381222063
Група: 91
Специалност: Киберсихурност

Научен ръководител:
Гл. Ас Маринов

София, 2026 г.

Съдържание

Задание ..... **Error! Bookmark not defined.**
Декларация за авторство ..... **Error! Bookmark not defined.**
Съдържание 1
УВОД 4
1. Изследване на съществуващи решения. Цели и задачи на приложението 5
1.1 Анализ на тематичната област. 5
1.2 Съществуващи решения 6
- Spotify 6
- Apple Music 6
- YouTube Music 6
1.3 Анализ на съществуващата система 7
1.4 Цели и задачи 8
Описание на използваните технологии. 10
2.1 Архитектура на системата 10
2.2 Технологии за разработка на сървърната част 12
- ASP.NET Core 12
- Entity Framework Core 12
- ASP.NET Identity 13
- JWT Authentication 13
- AutoMapper 13
- Swagger (OpenAPI) 14
- Unit Testing и интеграционно тестване 14
2.3 Система за управление на базата данни (Microsoft SQL Server) 14
- Релационен модел и структура на данните 15
- Нормализация на данните 15
- Роля на базата данни в системната архитектура 15
- Сигурност на данните 16
2.4 Технологии за разработка на клиентската част (Angular) 16
- TypeScript 16
- Компонентна архитектура 16

- RxJS и реактивно програмиране 17
- Routing и навигация 17
- Комуникация със сървърната част 17
- Обща роля на клиентската част 17

2.5 Извод 17

Проектиране 19

3.1 Архитектурно проектиране 19

3.2 Проектиране на базата данни 20

3.3 Проектиране на сървърната част 23

- Services 23
- Controllers 24
- DTO модели 24
- REST API 24
- JWT удостоверяване 24
- AutoMapper 24

3.4 Проектиране на клиентската част 25

- Angular модули 25
- Components (Компоненти) 25
- Routing (Маршрутизация) 26
- Guards (Защити) 26
- HTTP Interceptors 26

3.5 Проектиране на защитните механизми 26

- 2FA (Two-Factor Authentication) 27
- IP Filtering 27
- Email известяване 27
- Roles & Permissions 27
- Защита на API endpoints 27

Реализация 28

4.1 Backend реализация 28

4.2 Frontend реализация 28

4.3 Основни функционалности 28

4.4 Интеграция клиент–сървър 29

Тестване на кода (unit testing) 30
5.1 Тестове за CRUD операции с база данни 30
5.2 Тестове на контролери 30
5.3 Допълнителни тестове на бизнес логика 31
5.4 Подход към тестването и избор на методология 31
5.5 Представяне на тестовете 31
Киберсигурност 32
6.1 Автентикация чрез JWT (JSON Web Token) 32
6.2 Авторизация и Roles & Permissions модел 32
6.3 Защита на API endpoints 32
6.4 IP Filtering 33
6.5 Двуфакторна автентикация (2FA) 33
6.6 Email известяване при сигурностни събития 34
Ръководство на потребителя. 35
7.1 Конфигурация на базата данни 35
7.2 Създаване на база данни 35
7.3 Инициализация на тестови данни 35
7.4 Тестови потребители 36
Заключение (1 стр.) 48
Литература 50
Приложения 51
10.1 CategoryService ..... **Error! Bookmark not defined.**
10.2 CategoryServiceTests ..... **Error! Bookmark not defined.**
10.3 CategoryController ..... **Error! Bookmark not defined.**
10.4 CategoryControllerTests ..... **Error! Bookmark not defined.**

УВОД

С развитието на интернет технологиите онлайн платформите за публикуване и слушане на музика се превърнаха в основно средство за разпространение на музикално съдържание и взаимодействие между изпълнители и потребители. Наред с разширяването на техните функционалности, все по-голямо значение придобива и защитата на потребителските акаунти и съхраняваната информация.

Настоящата дипломна работа е посветена на разработването и надграждането на уеб платформа за публикуване и слушане на музика. Системата предоставя функционалности за регистрация и удостоверяване на потребители, публикуване и възпроизвеждане на музикални произведения, управление на плейлисти и албуми, коментиране, харесване на съдържание, последване на потребители и проследяване на историята на слушанията.

Основен акцент в разработката е внедряването на механизми за повишаване на информационната сигурност. Реализирани са двуфакторна автентикация (2FA), уведомяване по електронна поща при вход от нов IP адрес и система за филтриране на достъпа чрез потребителски правила за IP адреси. Тези функционалности повишават нивото на защита срещу неоторизиран достъп и предоставят на потребителите по-голям контрол върху сигурността на техните акаунти.

Приложението е разработено чрез използване на ASP.NET Core Web API, Angular, Entity Framework Core [3] и Microsoft SQL Server, като архитектурата следва принципите на Layered Architecture и Clean Architecture. Системата е проектирана с акцент върху модулността, разширяемостта и тестируемостта, което позволява лесна поддръжка и бъдещо развитие.

За гарантиране на качеството и надеждността на разработеното приложение са реализирани автоматизирани unit и integration тестове чрез xUnit [4] и MyTested.AspNetCore.Mvc [5]. Това позволява валидиране на бизнес логиката и правилното функциониране на отделните компоненти, както и навременно откриване на евентуални грешки.

В дипломната работа са разгледани анализът на предметната област, използваните технологии, архитектурното проектиране, програмната реализация на системата и разработените механизми за сигурност. Представени са също резултатите от проведеното тестване, които потвърждават коректната работа на реализираните функционалности.

1. Изследване на съществуващи решения. Цели и задачи на приложението

С нарастването на популярността на онлайн услугите и увеличаването на броя на регистрираните потребители, разработването на надеждни и сигурни уеб приложения се превръща във все по-значима задача. Съвременните информационни системи съхраняват голям обем лични данни и предоставят множество функционалности, което ги прави потенциална цел на различни кибератаки и опити за неоторизиран достъп. Поради тази причина сигурността вече не се разглежда като допълнителна функционалност, а като основен елемент при проектирането и разработването на всяко модерно уеб приложение.

Музикалните платформи не правят изключение. Освен че трябва да осигуряват бърз достъп до мултимедийно съдържание и удобен потребителски интерфейс, те следва да гарантират защита на потребителските акаунти, сигурно управление на личните данни и надежден контрол върху достъпа до различните ресурси на системата. Реализирането на подобни механизми е от съществено значение както за крайните потребители, така и за администраторите на платформата, тъй като повишава доверието в предоставяната услуга и намалява риска от злоупотреби.

Разработването на софтуерно решение, което съчетава функционалност, добра производителност и високо ниво на информационна сигурност, има важно практическо значение. Подобни системи намират приложение както в реални бизнес среди, така и като пример за прилагане на съвременни технологии и добри практики при проектирането на уеб приложения. Използването на многослойна архитектура, автоматизирано тестване и механизми за защита на потребителските акаунти допринася за създаването на надежден, разширяем и лесен за поддръжка софтуерен продукт.

Мотивът за разработването на настоящата дипломна работа е създаването и усъвършенстването на музикална платформа, която не само предоставя възможности за публикуване и слушане на музикално съдържание, но и интегрира съвременни механизми за защита на потребителите. По този начин се демонстрира практическото приложение на знания в областта на уеб разработката, базите данни, софтуерното инженерство и информационната сигурност при изграждането на цялостна информационна система.

1.1 Анализ на тематичната област.

Развитието на онлайн услугите в музикалната индустрия променя начина, по който потребителите достъпват и използват музикално съдържание. Увеличеният брой дигитални платформи води до по-високи изисквания към удобство, бързина и персонализация на услугите.

Това засяга както крайните потребители, които търсят лесен достъп и откриване на нова музика, така и артистите и компаниите, които се нуждаят от ефективно

разпространение и анализ на аудиторията. В същото време нараства значението на надеждни софтуерни решения, които могат да обработват големи обеми данни и да осигуряват сигурност и добро потребителско изживяване.

Поради това разработването на платформи за музикално съдържание е актуално и полезно, тъй като подпомага както потребителите, така и създателите на музика, като подобрява достъпа, организацията и взаимодействието с дигиталното съдържание.

1.2 Съществуващи решения

В настоящия раздел се разглеждат няколко съществуващи платформи за музикално съдържание, като се анализират техните основни характеристики, предимства и недостатъци. Следва да се отбележи, че тези системи са разработени от големи международни компании с ресурси и екипи от инженери, както и с утвърдено пазарно присъствие и значителни финансови възможности. За разлика от тях, настоящият проект представлява дипломна разработка, чиято цел е демонстрация на функционалност и архитектурни решения, а не конкуренция на индустриално ниво.

- **Spotify**

Spotify – една от най-популярните музикални платформи в света, предлагаща стрийминг на музика, персонализирани препоръки и социални функции. Предимствата ѝ включват изключително богата библиотека и добре развити алгоритми за препоръка. Недостатък е зависимостта от платен абонамент за пълна функционалност. Източник: https://www.spotify.com

- **Apple Music**

Apple Music – услуга на Apple, интегрирана в техните устройства и екосистема. Отличава се с високо качество на аудио стрийминга и силна интеграция с хардуера на компанията. Ограничение е по-слабата достъпност на някои платформи извън Apple средата.
Източник: https://www.apple.com/apple-music

- **YouTube Music**

YouTube Music – платформа на Google, която комбинира официална музика с потребителско съдържание от YouTube. Основно предимство е огромното разнообразие от съдържание, включително лайв изпълнения и ремикси. Недостатък е по-неконсистентното качество на съдържанието в сравнение с конкурентите. Източник: https://music.youtube.com

1.3 Анализ на съществуващата система

Настоящата дипломна работа представлява надграждане на вече разработена уеб платформа за публикуване и слушане на музика. Първоначалната версия на системата е разработена преди няколко години с цел участие в състезание по информационни технологии, като основната ѝ цел е била да демонстрира възможностите за изграждане на цялостна уеб платформа за управление и споделяне на музикално съдържание.

Съществуващата реализация предоставя богата функционалност както за крайните потребители, така и за администраторите на системата. Платформата позволява регистрация на нови потребители, удостоверяване чрез вход в системата и управление на потребителските профили. Реализирана е система за роли и права за достъп (Permissions), чрез която на всеки потребител могат да бъдат предоставяни конкретни разрешения за използване на различните функционалности на приложениетоПо този начин се реализира механизъм за контрол на достъпа, базиран на роли и индивидуални разрешения, който повишава нивото на информационна сигурност чрез ограничаване на достъпа до системните ресурси съобразно предоставените права.

В системата е реализирано управление на музикално съдържание, което включва организиране на песните по категории, качване на нови музикални произведения и тяхното възпроизвеждане директно през уеб приложението. Потребителите могат да създават собствени плейлисти и музикални албуми, като по този начин организират предпочитаното от тях съдържание според личните си предпочитания.

Допълнително платформата предлага механизъм за взаимодействие между потребителите и публикуваното съдържание. Реализирана е възможност за харесване и премахване на харесване на музикални произведения, което позволява оценяване на публикуваните песни. Системата поддържа и коментари към всяка песен, както и подкоментари, чрез които могат да се водят дискусии между потребителите. Осигурена е възможност за харесване на коментари, което допълнително подпомага социалното взаимодействие в платформата.

Социалните функционалности включват разглеждане на потребителски профили и възможност за последване на други потребители. По този начин се създава мрежа от връзки между изпълнители и слушатели, която улеснява откриването на ново музикално съдържание и следенето на активността на предпочитани автори. За всеки потребител се съхранява и история на прослушаните песни, което позволява лесен достъп до вече възпроизвежданото съдържание.

1.4 Цели и задачи

Въпреки богатата функционалност на първоначалната версия, анализът на системата показва, че нейните механизми за информационна сигурност могат да бъдат значително подобрени. Съществуващата реализация разполага със стандартен механизъм за удостоверяване чрез потребителско име и парола, но не предоставя допълнителни средства за защита срещу компрометиране на потребителските акаунти или неоторизиран достъп.

Основната цел на настоящата дипломна работа е разширяване на съществуващата уеб платформа за публикуване и слушане на музика чрез внедряване на съвременни механизми за повишаване на информационната сигурност. Макар първоначалната версия на системата да предоставя богата функционалност за управление на потребители, музикално съдържание и социално взаимодействие, нейният механизъм за удостоверяване се основава единствено на използването на потребителско име и парола. Този подход е широко разпространен, но не осигурява достатъчно ниво на защита срещу съвременните заплахи, като компрометиране на потребителски акаунти, кражба на идентификационни данни или неоторизиран достъп до системата.

В тази връзка основната цел на дипломната работа е да бъдат анализирани съществуващите механизми за защита на достъпа до уеб приложения и въз основа на този анализ да бъдат разработени и интегрирани допълнителни функционалности, които да повишат нивото на сигурност на платформата, без да нарушават удобството при работа на крайните потребители.

Една от основните задачи е реализирането на механизъм за двуфакторна автентикация (Two-Factor Authentication – 2FA). Чрез него процесът на удостоверяване ще изисква не само валидно потребителско име и парола, но и допълнителен еднократен код за потвърждение. По този начин се осигурява второ ниво на защита, което значително намалява вероятността за успешен неоторизиран достъп дори при компрометиране на потребителската парола.

Следваща задача е разработването на механизъм за уведомяване на потребителите при вход в системата от нов или непознат IP адрес. При установяване на подобно събитие платформата ще изпраща автоматично известие чрез електронна поща, съдържащо информация за извършения вход. Това позволява на потребителя своевременно да бъде информиран при потенциален опит за неправомерен достъп до неговия профил и при необходимост да предприеме подходящи действия за защита на акаунта си.

Като допълнителна защитна функционалност ще бъде разработена система за филтриране на достъпа по IP адреси (IP Access Filtering). Чрез нея всеки потребител ще може самостоятелно да дефинира списък с разрешени или забранени IP адреси и мрежови диапазони, от които може да бъде осъществяван достъп до неговия профил. При опит за

вход от адрес, който не отговаря на зададената политика за достъп, системата автоматично ще откаже удостоверяването. По този начин се реализира допълнителен механизъм за ограничаване на достъпа, който е особено подходящ при използване на системата от фиксирани местоположения или корпоративни мрежи.

Друга важна задача е интегрирането на разработените механизми със съществуващата архитектура на платформата, без да се нарушава нейната функционалност и производителност. Новите компоненти трябва да работят съвместно с вече реализираната система за удостоверяване, управление на роли и права за достъп, както и с останалите функционални модули на приложението. По този начин ще се запази съществуващата логика на системата, като същевременно ще бъде повишено нивото на защита на потребителските акаунти.

За постигане на поставената цел се дефинират следните основни задачи:

- анализ на съществуващата архитектура и механизмите за удостоверяване на платформата;
- анализ на съвременните методи за защита на потребителските акаунти в уеб приложения;
- проектиране и разработване на механизъм за двуфакторна автентикация;
- реализиране на автоматично известяване чрез електронна поща при вход от нов или непознат IP адрес;
- разработване на система за контрол на достъпа чрез IP Access Filtering;
- интегриране на новите защитни механизми към съществуващата платформа;
- тестване и анализ на разработените функционалности с цел оценка на тяхната ефективност и влияние върху сигурността на системата.

Изпълнението на поставените цели и задачи ще доведе до съществено повишаване на нивото на информационна сигурност на разработената платформа. В резултат ще бъде изградена по-надеждна система, която предоставя не само функционалности за публикуване и слушане на музика, но и съвременни механизми за защита на потребителските акаунти, ограничаване на неоторизирания достъп и своевременно уведомяване при съмнителна активност.

ОПИСАНИЕ НА ИЗПОЛЗВАНИТЕ ТЕХНОЛОГИИ

В настоящата глава е представен обзор на основните технологии, използвани при разработването на платформата за публикуване и слушане на музика. Изборът на използваните програмни средства и технологии е съобразен с изискванията за изграждане на съвременно уеб приложение, което да осигурява висока производителност, надеждност, сигурност и възможност за бъдещо разширяване. Разгледани са архитектурата на системата, технологиите за разработка на сървърната и клиентската част, както и използваната система за управление на базата данни. За всяка технология са представени нейните основни характеристики, причините за нейния избор и ролята ѝ в реализацията на разработената информационна система.

2.1 Архитектура на системата

Разработената платформа за публикуване и слушане на музика е реализирана като многослойно уеб приложение (Layered Architecture), при което отделните компоненти са ясно разделени според тяхната функционалност и отговорности. Този архитектурен подход позволява по-добра поддръжка, мащабируемост и възможност за бъдещо разширяване на системата, включително интегриране на допълнителни модули за сигурност и нови функционалности.

В основата на системата стои релационна база данни, реализирана чрез Microsoft SQL Server. Тя служи за съхранение на всички основни обекти в приложението, включително потребители, роли и права за достъп, музикално съдържание, категории, плейлисти, албуми, коментари и история на потребителската активност. Релационният модел на базата данни позволява ясно дефиниране на връзките между отделните таблици чрез първични и външни ключове, което гарантира цялостност и консистентност на данните. Освен това се използват индекси за оптимизация на заявките и повишаване на производителността при работа с големи обеми информация.

Над слоя за съхранение на данни е реализирана сървърната част на приложението, изградена чрез ASP.NET Core Web API. Тя представлява основния бизнес слой на системата и отговаря за обработката на всички заявки, идващи от клиентската част. В този слой се имплементира бизнес логиката на приложението, включително управление на потребителите, обработка на музикални файлове, социални взаимодействия и механизми за сигурност. За достъп до базата данни се използва ORM технологията Entity Framework Core [3], която позволява работа с данните чрез обектно-ориентиран подход и LINQ заявки, без необходимост от директно писане на SQL команди.

За удостоверяване и оторизация на потребителите в системата е внедрен механизъм, базиран на JWT (JSON Web Token). При успешен вход в системата се генерира токен, който съдържа информация за потребителя и неговите права за достъп. Този токен се използва при всяка последваща заявка към сървъра за валидиране на

идентичността и разрешенията на потребителя. Допълнително в сървърния слой са реализирани функционалности за качване и съхранение на файлове, както и изпращане на електронна поща чрез SMTP протокол, което се използва за уведомления при събития, свързани със сигурността на акаунта.

Клиентската част на системата е разработена като Single Page Application (SPA) с помощта на Angular. Тя осигурява потребителския интерфейс и визуализацията на всички функционалности на платформата. Чрез нея потребителите могат да се регистрират, да влизат в системата, да разглеждат и слушат музика, да създават плейлисти, да взаимодействат със съдържание и други потребители, както и да управляват своя профил. Клиентската част е структурирана модулно, което позволява разделяне на функционалностите и по-лесна поддръжка на кода. За управление на състоянието и комуникация със сървърната част се използват HTTP заявки към REST API интерфейса.

Комуникацията между клиентската и сървърната част се осъществява чрез REST архитектурен стил. Всяка функционалност на системата е реализирана чрез отделни HTTP endpoints, които обработват съответните заявки и връщат данни във формат JSON. Това осигурява независимост между двете части на приложението и позволява бъдещо разширяване, включително интеграция на мобилни приложения или други външни клиенти. Обменът на данни е защитен чрез използване на JWT токени, които се изпращат с всяка заявка към сървъра.

По този начин е реализирана многослойна архитектура, която ясно разделя отговорностите между отделните компоненти на системата. Това води до по-висока сигурност, по-лесна поддръжка и възможност за мащабиране на платформата при бъдещо развитие.

В допълнение към слоестата структура, системата е проектирана съобразно принципите на Clean Architecture и принципите за разделяне на отговорностите (Separation of Concerns). Това позволява ясно разграничаване между бизнес логиката, слоя за достъп до данни и представителния слой, като по този начин се минимизира зависимостта между отделните компоненти.

Прилагането на този подход осигурява висока степен на модулност и улеснява бъдещо разширяване и модификация на системата, без това да води до промени в останалите слоеве на приложението. Освен това архитектурният дизайн е ориентиран към тестируемост, като логиката на приложението е изолирана в отделни услуги (services), което позволява лесно създаване на unit и integration тестове.

По този начин системата е проектирана не само като функционално уеб приложение, но и като устойчив и разширяем софтуерен продукт, отговарящ на съвременните стандарти за софтуерно инженерство.

2.2 Технологии за разработка на сървърната част

Сървърната част на разработената платформа за публикуване и слушане на музика е реализирана чрез ASP.NET Core Web API, като основен технологичен стек за изграждане на бизнес логиката, обработка на заявки и управление на данните. Избраните технологии са ориентирани към висока производителност, модулност, тестируемост и възможност за лесно разширяване на системата, включително интеграция на допълнителни механизми за сигурност.

- **ASP.NET Core**



Фиг. 2.1 Лого ASP .NET CORE

ASP.NET Core представлява кросплатформена, високопроизводителна рамка за разработка на уеб приложения и API услуги. В настоящия проект тя е използвана за реализиране на RESTful Web API, което обслужва всички функционалности на клиентската част на системата.

Архитектурата на ASP.NET Core позволява прилагане на принципите на Dependency Injection, което води до слаба свързаност между отделните компоненти и улеснява тяхното тестване и поддръжка. Чрез middleware pipeline се осъществява контрол върху обработката на HTTP заявките, включително автентикация, логване, обработка на грешки и други междинни операции.

Допълнително ASP.NET Core предоставя висока степен на гъвкавост при изграждане на модулни системи, което е от съществено значение при разработка на приложения с разширена функционалност, каквато е музикалната платформа.

- **Entity Framework Core**



Фиг. 2.2 Лого entity framework core

Entity Framework Core (EF Core) [3] е ORM (Object-Relational Mapping) технология, която осигурява абстракция върху релационната база данни и позволява работа с данни чрез обектно-ориентиран модел.

В рамките на системата EF Core се използва за дефиниране на домейн моделите и управление на връзките между тях. Чрез LINQ заявки се извършват операции по създаване, четене, обновяване и изтриване на данни (CRUD), без необходимост от директно писане на SQL.

Допълнително EF Core поддържа миграции на база данни, което позволява контролирано управление на структурата на базата данни и нейното еволюиране във

времето. Това е особено важно при система, която се разширява с нови функционалности и изисква гъвкава поддръжка на схемата.

- **ASP.NET Identity**



Фиг. 2.3 Лого ASP.NET Identity

ASP.NET Identity е система за управление на потребители, автентикация и авторизация, която предоставя готова инфраструктура за работа с потребителски акаунти.

В настоящата платформа тя се използва за управление на регистрации, вход в системата, криптиране на пароли и управление на роли и права за достъп. Чрез механизма за роли и claims се осигурява детайлен контрол върху достъпа до различните функционалности на приложението.

Използването на ASP.NET Identity значително повишава нивото на сигурност, тъй като включва вградени механизми за защита срещу често срещани атаки, като brute-force атаки и неправомерен достъп до потребителски акаунти.

- **JWT Authentication**

За удостоверяване на потребителите и защита на API endpoints е използван JSON Web Token (JWT) базиран механизъм за автентикация.

При успешен вход в системата се генерира подписан токен, който съдържа информация за потребителя и неговите права за достъп. Този токен се изпраща от клиента при всяка заявка към сървъра и се използва за валидация на идентичността и разрешенията.

JWT подходът позволява изграждане на stateless архитектура, при която сървърът не съхранява сесийна информация, което повишава скалируемостта и производителността на системата.

- **AutoMapper**



Фиг. 2.4 Лого AutoMapper

AutoMapper е библиотека, използвана за автоматизирано преобразуване между различни обектни модели, най-често между домейн модели и Data Transfer Objects (DTO).

В проекта тя се използва за разделяне на вътрешната структура на данните от структурата, която се предоставя към клиентската част. По този начин се осигурява по-висока сигурност, като се избягва директното излагане на вътрешни модели към външния свят, както и се намалява количеството на ръчно писан код.

- **Swagger (OpenAPI)**



Фиг. 2.5 Лого Swagger

За документиране и проверка на функционалността на разработения Web API е използван Swagger. Той предоставя автоматично генерирана интерактивна документация, съдържаща информация за всички програмни интерфейси, техните параметри и върнатите резултати.

Освен като средство за документация, Swagger позволява директно тестване на отделните API методи, което улеснява процеса на разработка, проверка и последваща поддръжка на приложението.

- **Unit Testing и интеграционно тестване**



Фиг. 2.6 Лого xUnit

За осигуряване на качеството и надеждността на сървърната част са използвани unit тестове, реализирани чрез xUnit [4]. Те позволяват тестване на отделни компоненти от бизнес логиката в изолация, което е в съответствие с принципите на Clean Architecture и Dependency Injection.



Фиг. 2.7 Лого MyTested.AspNetCore.Mvc

Освен unit тестове, в проекта е използвана и библиотеката MyTested.AspNetCore.Mvc [5], разработена от Ивайло Кенов, която позволява писане на интеграционни тестове за ASP.NET Core приложения. Чрез нея могат да се тестват контролери, routing конфигурации, HTTP отговори и цялостното поведение на Web API слоя.

Комбинацията от unit и интеграционни тестове гарантира висока надеждност на системата и улеснява бъдещото ѝ развитие, като позволява безопасно въвеждане на нови функционалности и рефакториране на съществуващия код.

2.3 Система за управление на базата данни (Microsoft SQL Server)

За реализиране на персистентния слой на разработената платформа е използвана релационната система за управление на бази данни Microsoft SQL Server. Тя предоставя надеждна и високопроизводителна среда за съхранение и обработка на структурирани

данни, като същевременно поддържа транзакционност, цялостност на данните и механизми за оптимизация на заявки.

Изборът на релационна база данни е обусловен от характера на системата, която изисква ясно дефинирани връзки между различните домейн обекти като потребители, роли, музикално съдържание, категории, плейлисти, албуми и потребителска активност. Чрез използване на релационен модел се осигурява консистентност на данните и възможност за ефективно управление на сложни зависимости между отделните таблици.

- **Релационен модел и структура на данните**

Базата данни е проектирана на принципите на релационния модел, при който данните са организирани в таблици, свързани помежду си чрез първични и външни ключове. Всяка основна същност в системата е представена чрез отделна таблица, като връзките между тях отразяват бизнес логиката на приложението.

Основните таблици включват потребители, роли и права за достъп, музикални произведения, категории, плейлисти, албуми, коментари, подкоментари, харесвания и история на слушанията. Този модел позволява реализиране на богата функционалност, свързана както със съхранение на съдържание, така и със социални взаимодействия между потребителите.

- **Нормализация на данните**

При проектирането на базата данни са приложени принципите на нормализация с цел минимизиране на излишъка от данни и избягване на аномалии при операции по добавяне, редактиране и изтриване. Данните са структурирани така, че всяка информация да се съхранява на едно основно място, а връзките между таблиците да се осъществяват чрез референциална цялост.

Този подход гарантира по-висока степен на консистентност и улеснява бъдещо разширяване на схемата на базата данни, без да се нарушава съществуващата структура.

- **Роля на базата данни в системната архитектура**

Microsoft SQL Server играе централна роля в архитектурата на системата, като служи като единствен източник на истина (Single Source of Truth) за всички персистентни данни. Всички сървърни операции, свързани със създаване, обработка и извличане на информация, се базират на данните, съхранени в базата.

Чрез използване на Entity Framework Core [3] се осъществява абстракция върху директния достъп до базата, което позволява по-лесна поддръжка, по-висока сигурност и по-добра интеграция с останалите слоеве на приложението.

- **Сигурност на данните**

Особено внимание е отделено на защитата на данните, съхранявани в базата данни. Чувствителната информация, като пароли, не се съхранява в чист вид, а се обработва чрез криптографски алгоритми в сървърния слой преди запис в базата. Освен това достъпът до данните е ограничен чрез механизми за автентикация и авторизация, реализирани на ниво приложение.

По този начин се гарантира, че само оторизирани потребители и системни компоненти могат да извършват операции върху базата данни, което повишава общото ниво на информационна сигурност на платформата.

2.4 Технологии за разработка на клиентската част (Angular)



Фиг. 2.7 Лого Angular

Клиентската част на разработената платформа за публикуване и слушане на музика е реализирана като Single Page Application (SPA) чрез използване на Angular framework. Angular предоставя модерен подход за изграждане на динамични уеб приложения, като позволява разделяне на функционалностите в отделни модули и компоненти, което значително улеснява поддръжката и разширяването на системата.

Основната цел на клиентската част е да осигури интуитивен и бърз потребителски интерфейс, чрез който потребителите да взаимодействат с музикалното съдържание, да управляват своите профили и да използват социалните функционалности на платформата. Angular позволява ефективно управление на състоянието на приложението и динамично обновяване на потребителския интерфейс без необходимост от презареждане на страницата.

- **TypeScript**

Angular е базиран на TypeScript, който представлява надстройка на JavaScript, предоставяща статична типизация и по-добра структура на кода. Използването на TypeScript позволява ранно откриване на грешки по време на компилация, което повишава надеждността на приложението и улеснява неговото мащабиране.

- **Компонентна архитектура**

Клиентската част е изградена чрез компонентен подход, при който всяка функционалност на системата е реализирана като отделен компонент. Това включва компоненти за възпроизвеждане на музика, управление на плейлисти, потребителски профили, коментари и други функционални модули.

Компонентната архитектура позволява висока степен на преизползваемост на кода и ясно разделение на отговорностите в рамките на приложението.

- **RxJS и реактивно програмиране**

За управление на асинхронни операции и потоци от данни се използва библиотеката RxJS, която е интегрирана в Angular. Чрез нея се реализира реактивно програмиране, което позволява ефективна обработка на HTTP заявки, потребителски събития и динамични промени в състоянието на приложението.

Този подход е особено подходящ за уеб приложения, които работят с реално време и чести промени в данните, какъвто е случаят с музикалната платформа.

- **Routing и навигация**

Angular Router се използва за управление на навигацията в приложението. Чрез него се реализира логическо разделяне на различните функционални секции на платформата, като начална страница, профил на потребител, музикален каталог, плейлисти и административни панели.

Навигацията е реализирана по начин, който запазва поведението на SPA приложение, като се избягва презареждане на страницата и се осигурява плавно потребителско изживяване.

- **Комуникация със сървърната част**

Клиентската част комуникира със сървърната чрез REST API, като всички операции се извършват чрез **HTTP** заявки (**GET, POST, PUT, DELETE**). Данните се обменят във формат JSON, което осигурява лекота и бързина на комуникацията.

Този подход позволява пълно разделяне между клиентската и сървърната част на системата, което улеснява бъдещо разширяване, включително възможност за разработване на мобилно приложение, използващо същото API.

- **Обща роля на клиентската част**

Клиентската част представлява основния интерфейс между потребителя и системата, като предоставя всички функционалности на платформата в удобен и достъпен вид. Чрез използването на Angular се постига високо ниво на интерактивност, модулност и производителност, което е от съществено значение за системи, обработващи големи обеми динамично съдържание като музикални платформи.

2.5 Извод

Избраните технологии за разработка на системата са продиктувани от необходимостта за изграждане на модерно, мащабируемо и лесно поддържано уеб

приложение, което да осигурява добра производителност и удобство при работа. Използването на ASP.NET Core с Entity Framework [3] предоставя стабилна и сигурна backend архитектура, която позволява лесно управление на данните, интеграция с база данни и прилагане на добре структурирана бизнес логика. От страна на клиентската част, Angular (или Next.js, в зависимост от модула на системата) е предпочетен заради своя компонентно-базиран подход, който улеснява повторното използване на код, поддръжката на сложни потребителски интерфейси и добрата организация на големи приложения. Комбинацията от тези технологии позволява ясно разделение между frontend и backend, улеснява разширяемостта на системата и осигурява висока производителност, което ги прави най-подходящ избор за реализацията на проекта „MusicHub“.

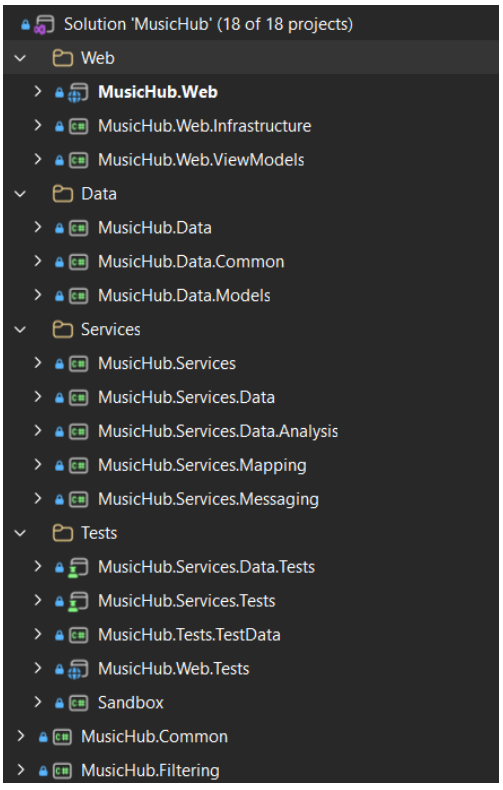
ПРОЕКТИРАНЕ

3.1 Архитектурно проектиране

При проектирането на платформата е избрана многослойна архитектура (Layered Architecture), която разделя приложението на отделни логически слоеве с ясно дефинирани отговорности. Основната цел на този подход е да се осигури ниска свързаност между отделните компоненти, висока модулност и възможност за независимо развитие на различните части на системата.

Архитектурата на приложението е съобразена и с принципите на Clean Architecture, при които бизнес логиката е отделена от слоя за достъп до данни, потребителския интерфейс и останалите инфраструктурни компоненти. Това позволява промени в една част на системата да бъдат извършвани с минимално влияние върху останалите модули и улеснява бъдещото разширяване на функционалността.

Проектът е организиран в няколко самостоятелни програмни библиотеки, всяка от които изпълнява конкретна роля в архитектурата на системата:



Фиг. 3.1 Файлова архитектура

MusicHub.Data реализира достъпа до базата данни, контекста на Entity Framework Core [3] и управлението на миграциите;

MusicHub.Data.Models – съдържа моделите, описващи структурата на данните и връзките между отделните обекти;

MusicHub.Services - реализира бизнес логиката на приложението чрез отделни услуги (services);

MusicHub.Services.Data - съдържа функционалностите за работа с данните и реализира операциите върху отделните домейн обекти;

MusicHub.Web - представлява Web API приложението, което предоставя програмния интерфейс към клиентската част;

MusicHub.Web.ViewModels – съдържа моделите за обмен на данни между сървърната и клиентската част;

MusicHub.Web.Infrastructure - включва общите инфраструктурни компоненти, използвани от Web приложението;

Tests - съдържа проектите за unit и интеграционно тестване на реализираната функционалност.

За осигуряване на ниска зависимост между отделните компоненти е използван механизмът Dependency Injection, който е основна част от ASP.NET Core. Всички услуги се регистрират в контейнера за зависимости и се предоставят автоматично на компонентите, които ги използват. По този начин отделните класове комуникират чрез интерфейси, а не чрез конкретни реализации, което улеснява поддръжката, разширяването и тестването на приложението.

При разработването на сървърната част е приложен асинхронен модел на програмиране чрез използването на ключовите думи async и await. Асинхронното изпълнение се използва при операции с по-голяма продължителност, като достъп до базата данни, обработка на файлове и изпращане на електронна поща. По този начин се избягва блокиране на изпълняващите нишки, подобрява се производителността на приложението и се осигурява възможност за едновременно обслужване на по-голям брой потребителски заявки.

Един от основните принципи при проектирането на системата е модулността. Всяка функционалност е реализирана в самостоятелен модул, което позволява независимо разработване, тестване и бъдещо разширяване. Например управлението на потребители, музикалното съдържание, плейлистите, коментарите и механизмите за сигурност са организирани като отделни логически компоненти, които взаимодействат чрез ясно дефинирани интерфейси.

Архитектурният модел е проектиран с фокус върху тестируемостта на приложението. Бизнес логиката е отделена от инфраструктурния слой, а използването на Dependency Injection позволява лесно заместване на реалните зависимости с тестови реализации. Това създава възможност за разработване както на unit тестове чрез xUnit [4], така и на интеграционни тестове чрез библиотеката MyTested.AspNetCore.Mvc [5], с което се гарантира коректността на реализираната функционалност и се улеснява последващото развитие на системата.

Използването на Layered Architecture, принципите на Clean Architecture, Dependency Injection, асинхронното програмиране и модулния подход водят до изграждането на гъвкава, разширяема и лесна за поддръжка система. Този архитектурен модел създава стабилна основа за внедряване на допълнителни функционалности, включително механизми за киберсигурност, които са основен акцент в настоящата дипломна работа.

3.2 Проектиране на базата данни

Проектирането на базата данни представлява един от най-важните етапи при разработването на платформата, тъй като тя осигурява съхранението и управлението на цялата информация, необходима за функционирането на системата. За реализацията е използван релационен модел, който позволява организиране на данните в ясно

дефинирани таблици и поддържане на връзки между тях посредством първични и външни ключове.

При разработването на приложението е приложен подходът Code First, предоставен от Entity Framework Core [3]. При този подход моделът на данните се проектира първоначално чрез C# класове (Entity Models), които описват отделните обекти и връзките между тях. Впоследствие структурата на базата данни се генерира автоматично чрез механизма на миграциите (Migrations), което осигурява синхрон между програмния код и схемата на базата данни. Използването на Code First улеснява последващото развитие на проекта, тъй като всяка промяна в моделите може лесно да бъде отразена в базата данни чрез създаване на нова миграция.

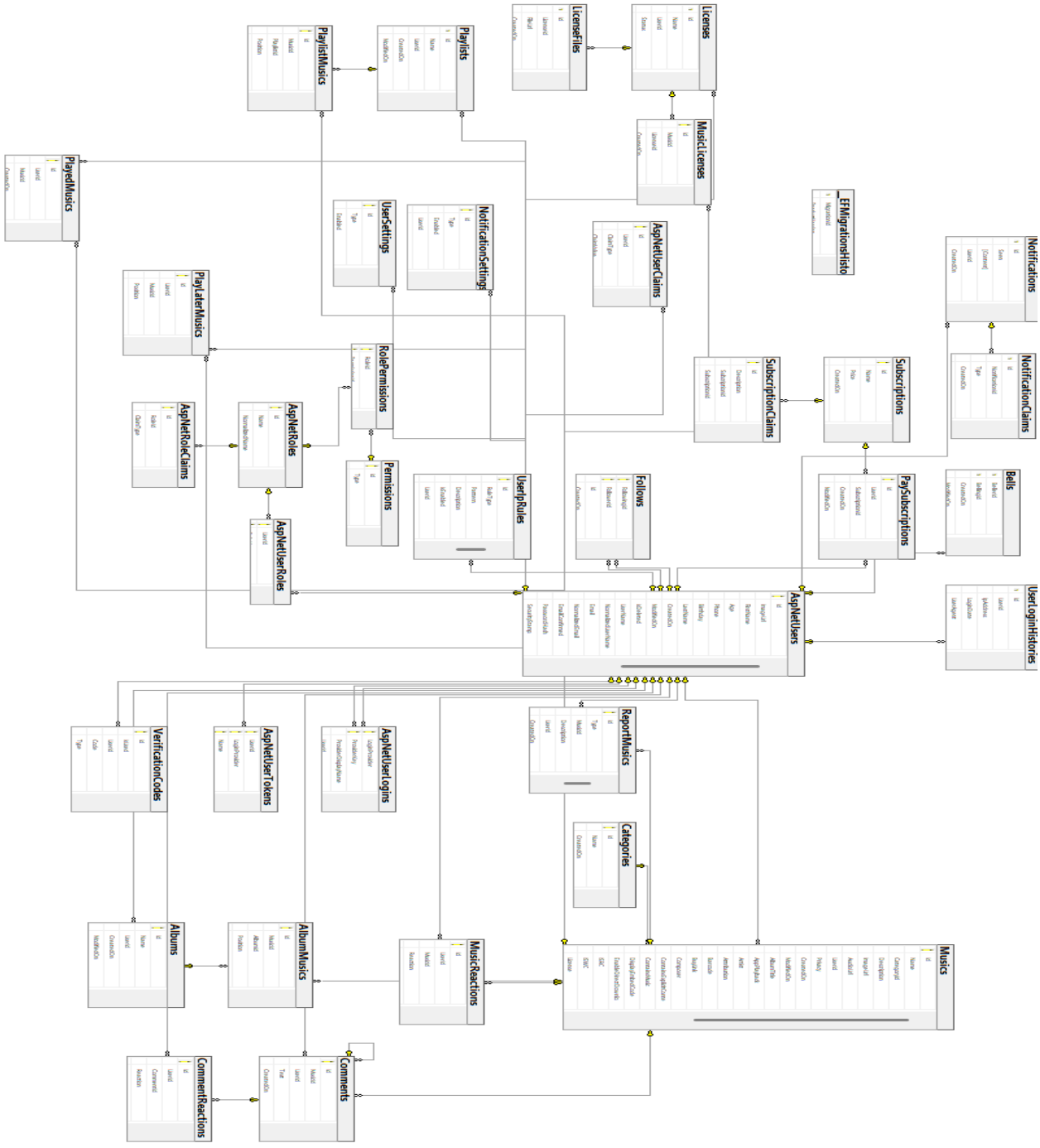
Фигура 3.2 представя ER диаграмата на разработената база данни. Тя визуализира основните таблици и връзките между тях, които реализират бизнес логиката на приложението.

Базата данни е организирана около няколко основни функционални области. Централно място заема таблицата AspNetUsers, която съхранява информация за потребителските акаунти и е основа за реализиране на удостоверяването и управлението на достъпа. Към нея са свързани множество допълнителни таблици, съдържащи информация за потребителските настройки, историята на входовете, последваните потребители, известията и останалите персонализирани данни.

Музикалното съдържание се съхранява в таблицата Musics, която съдържа информация за всяко публикувано произведение, включително неговите метаданни, собственик и настройки за достъп. С нея са свързани таблиците Categories, Albums, PlaylistMusics, MusicReactions, Comments, PlayedMusics и други, чрез които се реализират различните функционалности на платформата.

За управление на правата за достъп е използвана инфраструктурата на ASP.NET Identity, разширена с допълнителни таблици като Permissions, RolePermissions и UserIpRules, чрез които се реализира по-гъвкава политика за авторизация и контрол на достъпа. Това позволява предоставяне на индивидуални разрешения за всеки потребител, независимо от неговата роля в системата.

Особено внимание е отделено на реализирането на функционалностите, свързани с информационната сигурност. За тази цел са проектирани специализирани таблици като UserLoginHistories, използвана за съхраняване на информация за извършените входове в системата, VerificationCodes, предназначена за съхраняване на кодовете за двуфакторна автентикация и потвърждение на различни операции, както и UserIpRules, която реализира механизма за филтриране на достъпа според IP адреса на потребителя.



Фиг. 3.2. ER диаграма на базата данни

Връзките между отделните таблици са реализирани чрез първични и външни ключове, като по този начин се гарантира референтната цялост на данните. При необходимост са реализирани връзки от тип „един към много“ и „много към много“, което позволява моделиране на сложните зависимости между потребители, музикални произведения и останалите обекти в системата.

С цел улесняване на разработката и тестването на приложението е реализирано автоматично начално попълване на базата данни (Database Seeding). Чрез специализирани класове (Seeders) при създаването на базата се добавят тестови потребители, роли, категории, разрешения и друга примерна информация, необходима за нормалното функциониране на системата по време на разработката и изпълнението на автоматизираните тестове. Този подход осигурява еднаква начална среда при всяко ново разгръщане на приложението и значително улеснява процесите по разработка и валидиране на реализираната функционалност.

Избраният модел на проектиране осигурява добра мащабируемост и възможност за бъдещо разширяване на системата. Благодарение на използването на Code First, миграции и автоматично инициализиране на данните, структурата на базата данни остава лесна за поддръжка, синхронизирана с програмния код и готова за интегриране на нови функционалности без необходимост от ръчна модификация на схемата.

3.3 Проектиране на сървърната част

Сървърната част на платформата е реализирана като RESTful Web API, разработено чрез ASP.NET Core. Основната ѝ задача е да обработва клиентските заявки, да реализира бизнес логиката на приложението, да осъществява достъп до базата данни и да осигурява сигурен обмен на информация между клиентската част и сървъра.

При проектирането е приложен принципът за разделяне на отговорностите (Separation of Concerns), като различните функционалности са организирани в отделни слоеве. По този начин всяка част от системата изпълнява конкретна задача, което улеснява разработката, поддръжката и бъдещото разширяване на приложението.

- **Services**

Бизнес логиката на системата е реализирана чрез отделни Service класове. Всеки модул на приложението разполага със собствена услуга, която съдържа логиката за обработка на данните и реализиране на съответната функционалност. По този начин контролерите не съдържат бизнес логика, а единствено приемат клиентските заявки, извикват необходимите услуги и връщат резултат към клиента.

Разделянето на бизнес логиката в самостоятелни услуги позволява многократното ѝ използване от различни части на приложението и улеснява създаването на unit тестове.

- **Controllers**

Контролерите представляват входната точка към сървърната част на приложението. Всеки контролер предоставя набор от HTTP методи, чрез които клиентската част може да извършва операции върху отделните функционални модули.

В проекта са реализирани контролери за управление на потребители, музикални произведения, албуми, плейлисти, коментари, известия и останалите функционалности на платформата. Освен тях са разработени и контролери, свързани с механизмите за сигурност, включително двуфакторната автентикация, управлението на IP правилата и обработката на потребителските сесии.

- **DTO модели**

За обмен на информация между клиентската и сървърната част се използват Data Transfer Objects (DTO). Тези модели съдържат единствено необходимите данни за конкретната операция и не разкриват вътрешната структура на домейн моделите.

Използването на DTO модели намалява количеството обменяни данни, подобрява сигурността на приложението и осигурява ясно разделение между вътрешната бизнес логика и външния програмен интерфейс.

- **REST API**

Комуникацията между Angular клиента и сървърната част се реализира чрез REST API. Всички функционалности на платформата са достъпни чрез HTTP заявки, като се използват стандартните методи GET, POST, PUT и DELETE.

Обменът на информация се извършва във формат JSON, което осигурява независимост между клиентската и сървърната част и позволява едно и също API да бъде използвано както от уеб клиента, така и от бъдещи мобилни или външни приложения.

- **JWT удостоверяване**

За защита на програмния интерфейс е реализирана автентикация чрез JSON Web Token (JWT). След успешно удостоверяване на потребителя сървърът генерира токен, съдържащ информация за неговата самоличност и предоставените му права за достъп. При всяка следваща заявка този токен се изпраща към сървъра, където се извършва неговата проверка преди изпълнение на заявената операция.

Този подход позволява изграждане на stateless архитектура и осигурява сигурен контрол върху достъпа до защитените ресурси на приложението.

- **AutoMapper**

За преобразуване между домейн моделите и DTO обектите е използвана библиотеката AutoMapper. Чрез нея автоматично се извършва преобразуването между различните типове обекти, което значително намалява необходимостта от ръчно копиране на свойства.

Използването на AutoMapper води до по-четим и лесен за поддръжка код, като същевременно намалява вероятността от допускане на грешки при обработката на данните.

3.4 Проектиране на клиентската част

Клиентската част на системата MusicHub е разработена с Angular и има за цел да осигури модерен, бърз и удобен потребителски интерфейс. Архитектурата е базирана на модулен подход, който позволява разделяне на функционалностите и по-лесна поддръжка и разширяване на приложението.

- **Angular модули**

Приложението е организирано в отделни Angular модули, като всеки модул обединява свързани функционалности. Този подход подобрява структурата на проекта и улеснява мащабирането.

Основните модули са:

- **CoreModule** – съдържа основни услуги като автентикация, HTTP interceptors и guards. Зарежда се веднъж при стартиране на приложението.
- **SharedModule** – включва общи компоненти, директиви и помощни функции, които се използват в различни части на системата.
- **AuthModule** – отговаря за регистрация и вход на потребители.
- **MusicModule** – съдържа функционалности, свързани с музиката, като списък с песни и детайлен изглед.
- **PlaylistModule** – управление на плейлисти.
- **UserModule** – потребителски профил и свързани с него данни.

Тази структура позволява ясна разделеност на отговорностите между отделните части на приложението.

- **Components (Компоненти)**

Потребителският интерфейс е изграден от множество Angular компоненти, като всеки компонент отговаря за конкретна част от визуализацията или функционалността.

Основните компоненти включват:

- компоненти за вход и регистрация на потребители
- компоненти за списък с музика и детайли за конкретна песен

- компоненти за любими песни
- компоненти за плейлисти
- компонент за потребителски профил

Всеки компонент съдържа три основни части: HTML шаблон, стилове и TypeScript логика. Това следва принципа на разделяне на отговорностите и улеснява повторното използване на код.

- **Routing (Маршрутизация)**

Навигацията в приложението се реализира чрез Angular Router, който позволява работа като Single Page Application (SPA). Това означава, че страниците се сменят динамично без презареждане на браузъра.

Част от модулите са конфигурирани с lazy loading, което подобрява началното време за зареждане и производителността на системата.

- **Guards (Защити)**

За контрол на достъпа до различните части на приложението се използват Angular guards. Те проверяват дали потребителят има право да достъпва даден маршрут.

Основните guards са:

AuthGuard – позволява достъп само на автентикирани потребители

RoleGuard – ограничава достъпа според ролята на потребителя

Тези механизми използват JWT токен за проверка на автентикацията и авторизацията.

- **HTTP Interceptors**

HTTP interceptors се използват за централизирана обработка на HTTP заявките. Основната им роля в системата е автоматичното добавяне на JWT токен към всички заявки към сървъра.

Освен това те могат да обработват глобално HTTP грешки, като например 401 Unauthorized, при което потребителят се пренасочва към страницата за вход. Това намалява дублирането на код и подобрява сигурността на приложението.

3.5 Проектиране на защитните механизми

За осигуряване на високо ниво на сигурност в системата **MusicHub** са внедрени няколко защитни механизма, които обхващат както автентикацията и достъпа на потребителите, така и защитата на API и чувствителните операции.

- **2FA (Two-Factor Authentication)**

Системата поддържа двуфакторна автентикация, която добавя допълнителен слой защита при вход. Освен стандартното потребителско име и парола, потребителят трябва да потвърди самоличността си чрез допълнителен код, изпратен по email или генериран от външен механизъм. Това значително намалява риска от неоторизиран достъп при компрометирани пароли.

- **IP Filtering**

В системата е внедрен механизъм за IP филтриране, който позволява ограничаване на достъпа до приложението или определени негови части само от разрешени IP адреси. Това е особено полезно за административни функции или вътрешни операции, като по този начин се намалява повърхността за атаки.

- **Email известяване**

Системата използва email известия за важни събития, свързани със сигурността на потребителския акаунт. Такива събития включват нов вход в профила, промяна на парола или активиране на ново устройство. Това позволява на потребителите своевременно да реагират при съмнителна активност

- **Roles & Permissions**

Достъпът до функционалностите в системата е контролиран чрез роли и права. Всеки потребител има определена роля (например User или Admin), като на базата на нея се определя кои операции може да извършва. Това осигурява ясно разделение на отговорностите и предотвратява достъп до административни функции от обикновени потребители.

- **Защита на API endpoints**

Всички чувствителни API endpoints са защитени чрез JWT базирана автентикация. Само валидни и автентикирани заявки имат достъп до ресурсите. Допълнително се използват authorization проверки на ниво endpoint, които валидират правата на потребителя спрямо заявената операция. Това гарантира, че всяка заявка е както автентикирана, така и оторизирана.

РЕАЛИЗАЦИЯ

Системата **MusicHub** е реализирана като модерно уеб приложение с разделена клиент-сървър архитектура. Клиентската част е разработена като Single Page Application (SPA) с Angular, а сървърната част предоставя REST API услуги.

Обмяната на данни между двете части се осъществява чрез HTTP заявки, като се използва JSON формат за пренос на информация. За сигурност на комуникацията е внедрен механизъм за автентикация чрез JWT (JSON Web Token), който гарантира защитен достъп до ресурсите на системата.

4.1 Backend реализация

Сървърната част е изградена като REST API, реализирано чрез контролери, които отговарят за основните функционални модули – музика, потребители и плейлисти. Всеки контролер предоставя набор от endpoints за извършване на CRUD операции.

Бизнес логиката е отделена в service слой, което позволява по-добра структура и лесна поддръжка на кода. Достъпът до базата данни е реализиран чрез Entity Framework Core [3], използвайки Code First подход за дефиниране на моделите и тяхното автоматично създаване в базата данни.

За сигурност се използва JWT автентикация, комбинирана с ролево базиран контрол на достъпа (Roles & Permissions), който определя правата на потребителите спрямо тяхната роля в системата.

4.2 Frontend реализация

Клиентската част е реализирана с Angular и е структурирана чрез модулен подход. Всеки модул отговаря за конкретна част от функционалността, като например музика, плейлисти, автентикация и потребителски профил.

UI логиката е разделена в компоненти, които съдържат отделно HTML шаблон, стилове и TypeScript логика. За комуникация със сървъра се използват Angular services, които инкапсулират HTTP заявките.

Навигацията в приложението се управлява чрез Angular Router, като се поддържа lazy loading за оптимизиране на зареждането. Допълнително са използвани guards за защита на маршрути и HTTP interceptors за автоматично добавяне на JWT токен към всички заявки.

4.3 Основни функционалности

Системата предоставя основни функционалности, свързани с управление на музикално съдържание и потребителски профили. Потребителите могат да се регистрират

и да влизат в системата, след което имат достъп до списък с музика, детайлен преглед на песни и възможност за добавяне към любими.

Допълнително е реализирана функционалност за създаване и управление на плейлисти, както и персонализиран потребителски профил. В зависимост от ролята на потребителя са налични и административни операции.

4.4 Интеграция клиент–сървър

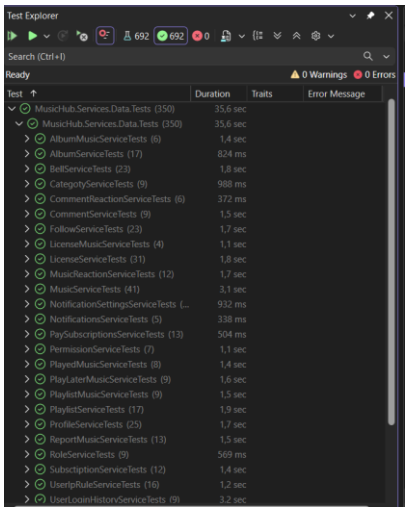
Комуникацията между Angular приложението и REST API е изцяло базирана на HTTP протокол. Клиентската част изпраща заявки към сървъра, който обработва логиката и връща резултатите във формат JSON.

JWT токенът се съхранява от клиента и се добавя автоматично към всяка заявка чрез HTTP interceptor. Това осигурява сигурна комуникация и контрол на достъпа до защитените ресурси в системата.

ТЕСТВАНЕ НА КОДА (UNIT TESTING)

В процеса на разработка на системата MusicHub е отделено значително внимание на тестването на функционалността с цел гарантиране на коректност, стабилност и надеждност на приложението. Общият брой реализирани unit тестове е 692, като те са разделени според тестваните слоеве и логика на системата.

5.1 Тестове за CRUD операции с база данни



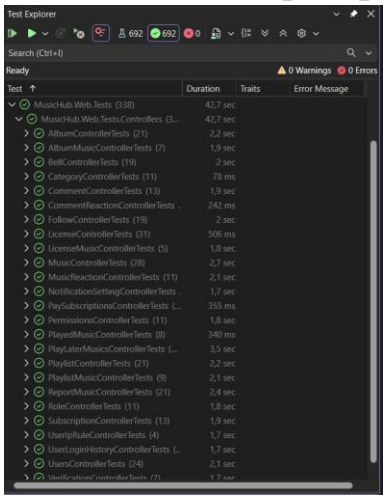
Фиг. 5.1. Резултати от тесовете върху бизнес логиката

Най-голямата част от тестовете – **350 на брой**, са свързани с CRUD операции върху базата данни. Те са реализирани с помощта на библиотеката xUnit [4] и използват подхода In-Memory Database, предоставен от Entity Framework Core [3].

Този подход позволява симулиране на реална база данни в паметта, без необходимост от външна зависимост. По този начин се постигат бързи и изолирани тестове, които проверяват коректността на бизнес операциите.

В рамките на тези тестове не се използва mock-ване на repository или services. Причината за този избор е стремежът към по-реалистично тестване на приложението, при което се валидира цялата верига от логика – от service слоя до достъпа до данни.

5.2 Тестове на контролери



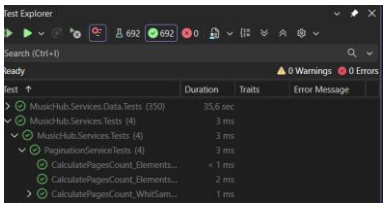
Фиг. 5.2. Резултати от тесовете върху контролерите

Общо 338 теста са предназначени за тестване на API контролерите. За тях е използвана библиотеката MyTested.AspNetCore.Mvc [5], разработена от Ивайло Кенов, която предоставя удобен и изразителен начин за тестване на ASP.NET Core контролери.

Библиотеката позволява симулиране на HTTP заявки и проверка на резултатите от контролерите по начин, близък до реалното поведение на API-то.

Част от тестовете са надградени с допълнителна логика спрямо базовите примери, предоставени от автора на библиотеката, като по този начин се постигат по-задълбочени проверки и по-високо ниво на покритие на функционалността.

5.3 Допълнителни тестове на бизнес логика



Фиг. 5.3. Резултати от тесовете върху друга бизнес логика в приложението

Останалите 4 теста са насочени към специфична бизнес логика в приложението, по-специално към pagination service. Тези тестове проверяват коректното разделяне на данни на страници и правилното изчисляване на броя елементи и страници.

5.4 Подход към тестването и избор на методология

В рамките на проекта е направен съзнателен избор да се използват реални интеграционни подходи чрез In-Memory база данни, вместо широко използваното mock-ване на зависимости. Този подход позволява по-висока степен на достоверност на тестовете, тъй като се проверява реалното взаимодействие между слоевете на приложението.

За разлика от mock-базираните unit тестове, този подход намалява риска от несъответствия между тестовата среда и реалната работа на системата, макар и да изисква повече време за изпълнение на тестовете.

5.5 Представяне на тестовете

С оглед на големия брой реализирани тестове (692), в рамките на настоящата разработка са подбрани и представени приблизително 10 примерни теста, които най-добре илюстрират използваните подходи, структурата на тестовете и основните сценарии на тестване.

Този подбор е направен с цел да се постигне баланс между пълнотата на проекта и неговата четимост.

КИБЕРСИГУРНОСТ

Киберсигурността представлява съществен елемент при проектирането и реализацията на системата MusicHub, тъй като приложението работи с потребителски данни, автентикация и различни нива на достъп до функционалности и ресурси. Основната цел на внедрените механизми за сигурност е да се гарантира защита от неоторизиран достъп, злоупотреба с ресурси и потенциални атаки към API слоя на системата.

Системата е проектирана с подход на многослойна защита (defense-in-depth), при който сигурността не се базира на един единствен механизъм, а на комбинация от няколко независими нива на защита, работещи съвместно.

6.1 Автентикация чрез JWT (JSON Web Token)

Основният механизъм за автентикация в системата е базиран на JWT (JSON Web Token). При успешно удостоверяване на потребителя се генерира токен, който съдържа информация за неговата идентичност и роли.

Този токен се изпраща с всяка последваща HTTP заявка към сървъра и се валидира автоматично от backend слоя. По този начин се гарантира, че достъп до защитени ресурси получават единствено автентикирани потребители.

Използването на JWT позволява също така безсесийна (stateless) архитектура, което улеснява скалирането на системата и намалява зависимостта от сървърно съхранени сесии.

6.2 Авторизация и Roles & Permissions модел

Освен автентикация, системата прилага и механизъм за авторизация, базиран на роли и права. Всеки потребител в системата притежава определена роля (например обикновен потребител или администратор), която определя неговите възможности в приложението.

Този модел позволява ясно разграничаване на отговорностите и достъпа до функционалности. Например административни операции като управление на потребители или модериране на съдържание са ограничени единствено до потребители с администраторски права.

По този начин се минимизира рискът от неоторизирани действия и се осигурява контролирана среда за работа на системата.

6.3 Защита на API endpoints

Всички чувствителни API endpoints в системата са защитени чрез комбинация от JWT автентикация и допълнителни проверки за авторизация. Всеки входящ request се

валидира, като първо се проверява наличието и валидността на токена, а след това и правата на потребителя спрямо конкретната операция.

Този двустепенен модел на проверка гарантира, че дори при наличен валиден токен, потребителят не може да извършва действия извън неговия обхват на права.

Допълнително, API архитектурата следва принципа на минимални права (least privilege), при който всеки endpoint предоставя единствено необходимия достъп за съответната операция.

6.4 IP Filtering

В системата е внедрен механизъм за IP филтриране, който позволява ограничаване на достъпа до определени функционалности или административни части на приложението само от предварително разрешени IP адреси. Реализацията е осъществена чрез **middleware компонент**, който обработва всяка входяща HTTP заявка и извършва проверка на IP адреса на клиента. Ако адресът не принадлежи към списъка с разрешени IP адреси, заявката се прекратява и достъпът до защитения ресурс се отказва.

Този подход е особено полезен при защита на чувствителни операции, като административни панели или вътрешни системни функции. Чрез ограничаване на достъпа още на ниво обработка на HTTP заявките се намалява повърхността за атака и се повишава общата сигурност на системата. Използването на middleware позволява проверката да се извършва централизирано за всички заявки, без необходимост логиката да се повтаря във всеки контролер или действие.

6.5 Двуфакторна автентикация (2FA)



Фиг. 6.1. Лого 2FA

Като допълнително ниво на защита е внедрена двуфакторна автентикация (2FA). Този механизъм изисква от потребителя, освен стандартните потребителско име и парола, да потвърди своята самоличност чрез еднократен код. В конкретната реализация този код се изпраща на електронната поща на потребителя при опит за вход.

След получаването на кода, потребителят трябва да го въведе в системата, за да завърши процеса по автентикация. По този начин се добавя допълнителен слой на защита, който значително намалява риска от неоторизиран достъп, дори при компрометирани идентификационни данни.

6.6 Email известяване при сигурностни събития

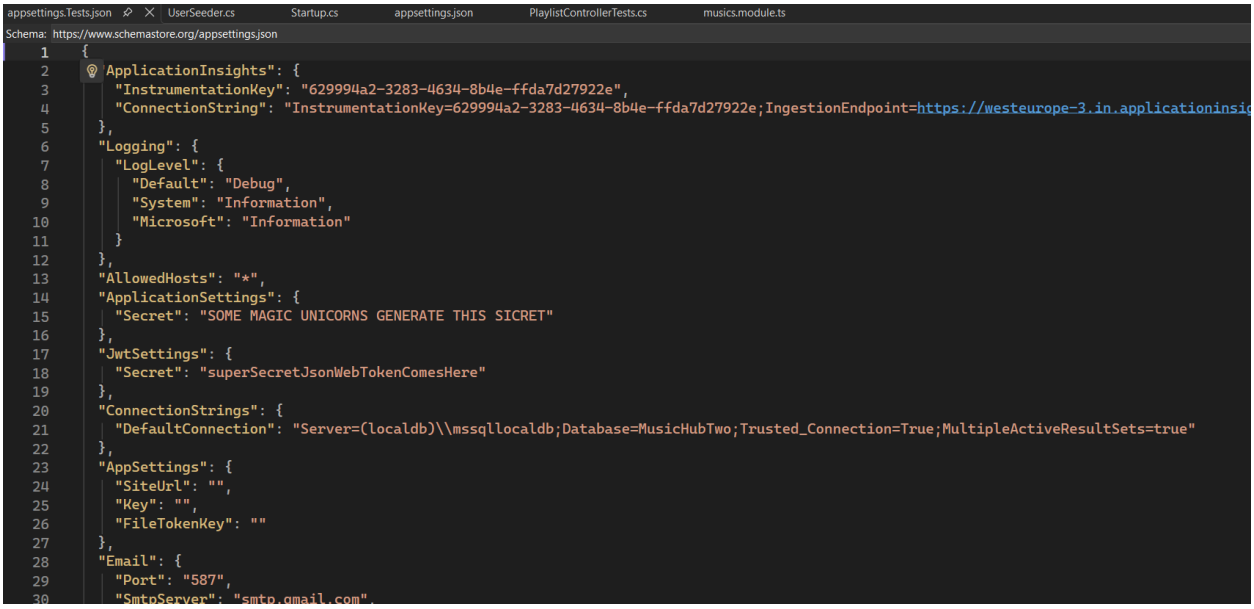
Системата разполага и с механизъм за **email известяване**, който информира потребителите при важни събития, свързани със сигурността на техния акаунт.

Такива събития включват успешен вход от ново устройство, промяна на парола или други чувствителни операции. Това позволява на потребителите своевременно да реагират при съмнение за неоторизиран достъп

РЪКОВОДСТВО НА ПОТРЕБИТЕЛЯ

7.1 Конфигурация на базата данни

Връзката към базата данни е дефинирана в проекта MusicHub.Web във файла appsettings.json. В този файл се намира connection string-ът, който определя параметрите за достъп до сървъра и базата данни.



Фиг. 7.1 Съдържание на appsettings.json

При необходимост от локално стартиране на проекта, този параметър може да бъде променен според конкретната среда (локален SQL Server или друг сървър).

7.2 Създаване на база данни

За създаване и инициализиране на базата данни се използва подходът Entity Framework Core Migrations [3]. След отваряне на проекта, в Package Manager Console се изпълнява следната команда:

- `Update-Database`

Тази команда създава необходимата структура в базата данни според дефинираните модели в приложението.

7.3 Инициализация на тестови данни

В проекта е реализиран seeder, който се изпълнява при стартиране на приложението (в Startup.cs). Неговата цел е автоматично да зареди примерни данни в системата, които улесняват тестването и демонстрацията на функционалностите.

Seeder-ът създава предварително дефинирани потребители с различни роли и права, което позволява тестване на различни сценарии в системата.

7.4 Тестови потребители

За целите на демонстрация и тестване в системата са налични следните потребители:

Роля	Потребителско име	Парола
SuperAdministrator	rootadmin	rootpass
Administrator	rootadmin2	rootpass
DataAnalyst	rootadmin3	rootpass
User	rootuser	rootpass
User	rootuser1	rootpass
User	rootuser2	rootpass
User	pesho	rootpass
User	gosho	rootpass

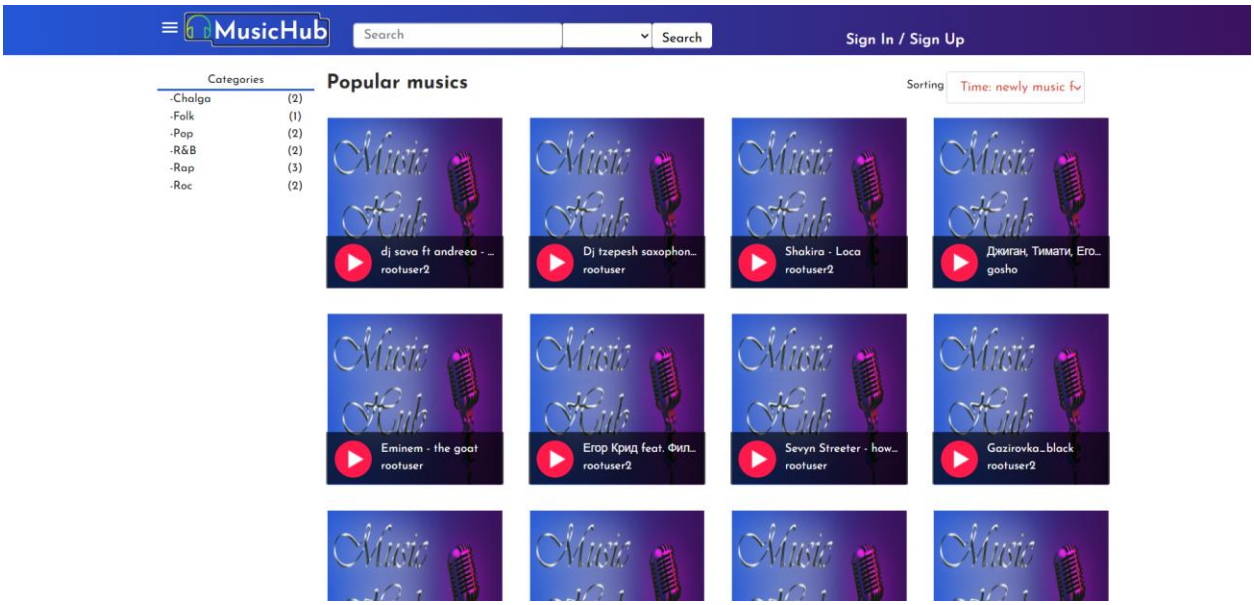
Таб. 6.1 Тестови потребители

Тези акаунти позволяват тестване на различните роли и функционалности в системата, включително административни и потребителски операции.

7.5 Използване на приложението

- **Начална страница**

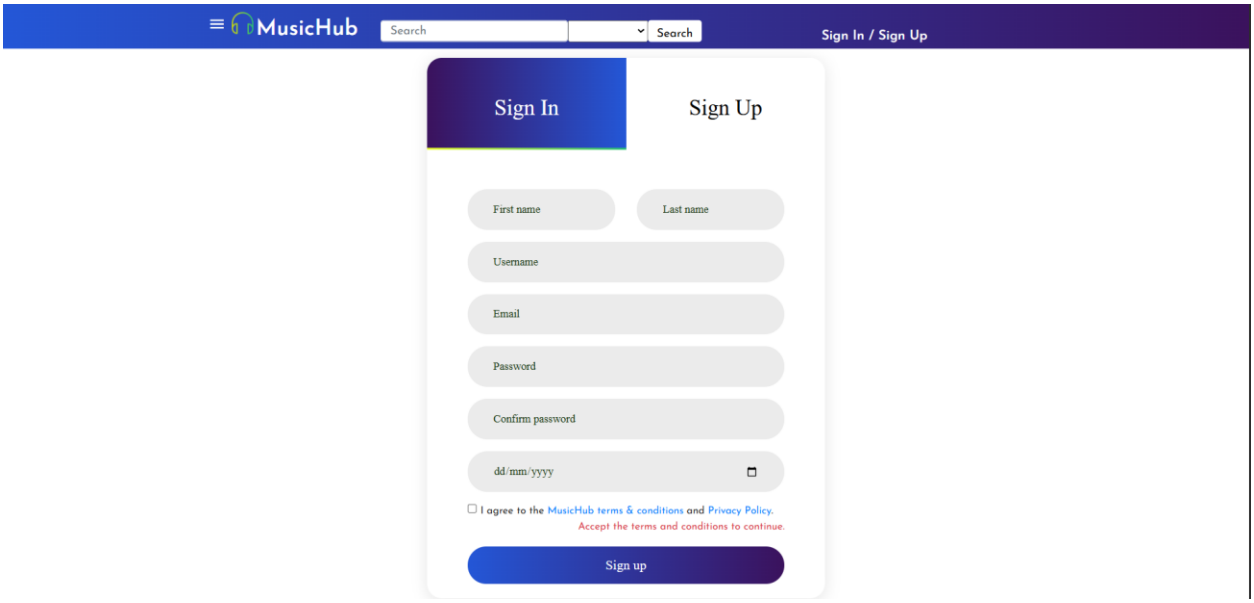
При отваряне на уеб приложението потребителят вижда началната страница, на която се визуализират всички публикувани музикални произведения. Зареждането на съдържанието е реализирано чрез механизъм за динамично извличане на данни (Infinite Scroll). При достигане до края на текущо заредените записи системата автоматично извлича следващата група песни от базата данни, без необходимост от презареждане на страницата. Това осигурява плавна навигация и по-добро потребителско изживяване.



Фиг. 7.2 Начален екран на приложението

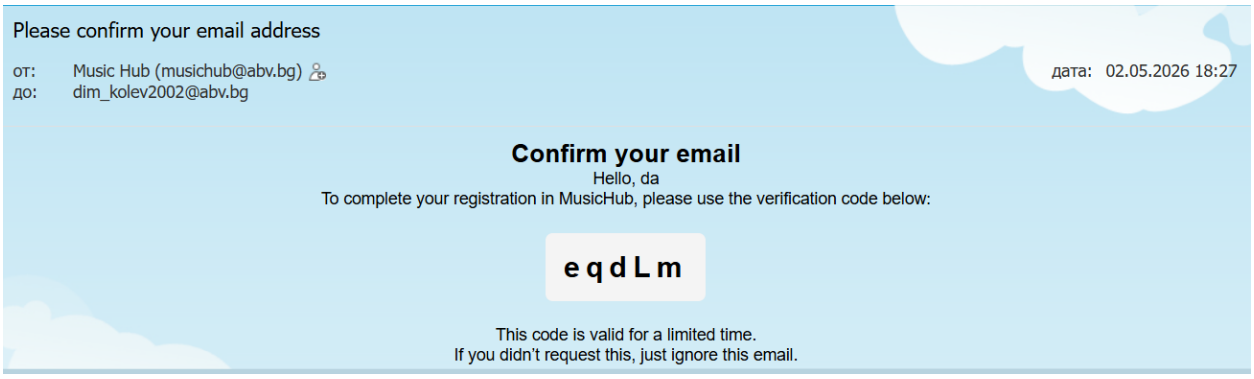
- **Регистрация на потребител**

За създаване на нов потребителски профил е необходимо да бъдат попълнени следните данни: име, фамилия, потребителско име, електронна поща, парола и дата на раждане. Преди завършване на регистрацията потребителят трябва да се съгласи с Общите условия на платформата, като маркира съответното поле. След попълване на всички задължителни данни регистрацията се извършва чрез натискане на бутона **Sign Up**.

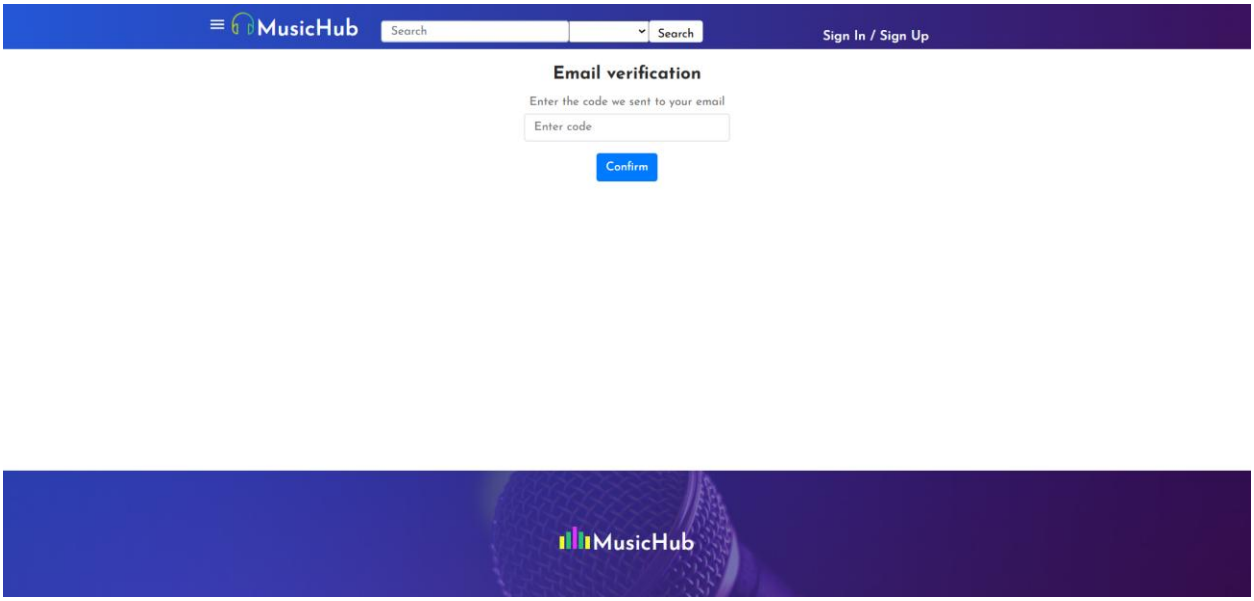


Фиг. 7.3 Форма за регистрация

След успешното създаване на профила системата изпраща съобщение за потвърждение на посочения електронен адрес. Потребителят трябва да отвори получения имейл и да потвърди своята регистрация чрез предоставената връзка, след което профилът се активира.



Фиг. 7.4 Съобщение за потвърждение на email

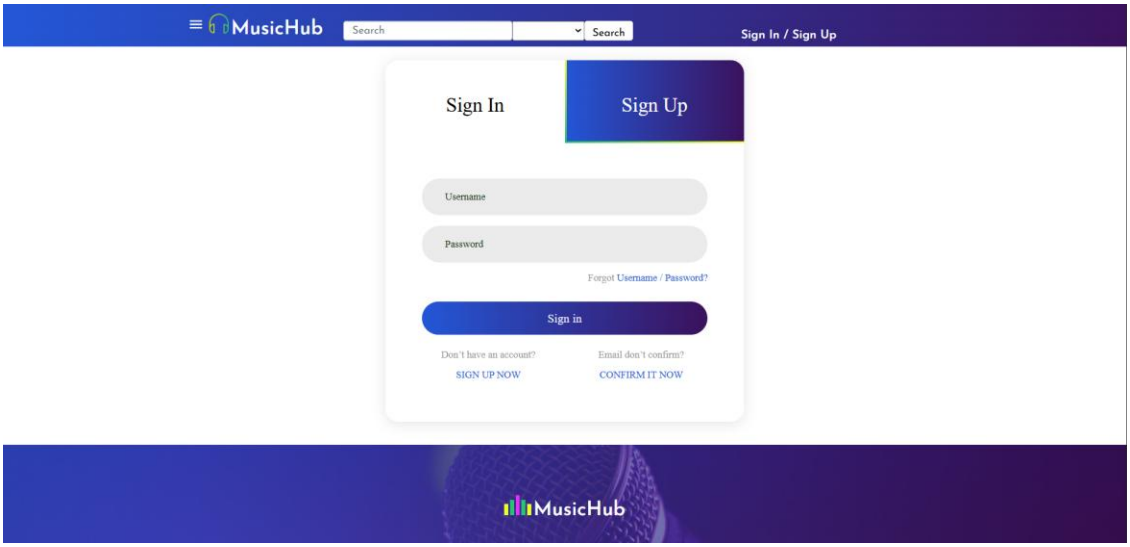


Фиг. 7.5 Въвеждане на код за потвърждение на email

За всеки новорегистриран потребител по подразбиране е активирана двуфакторна автентикация чрез електронна поща. При всяко влизане в системата на регистрирания имейл адрес се изпраща еднократен код за потвърждение, който трябва да бъде въведен, за да бъде завършен процесът на удостоверяване.

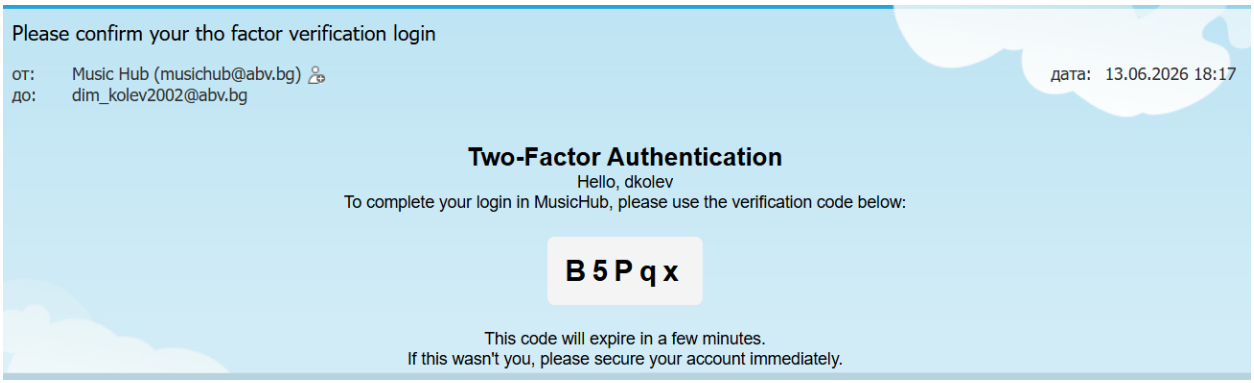
- **Вход в системата**

За достъп до платформата потребителят използва формата за вход, като трябва да въведе своето потребителско име и парола. След успешно валидиране на въведените данни системата стартира процес на двуфакторна автентикация.

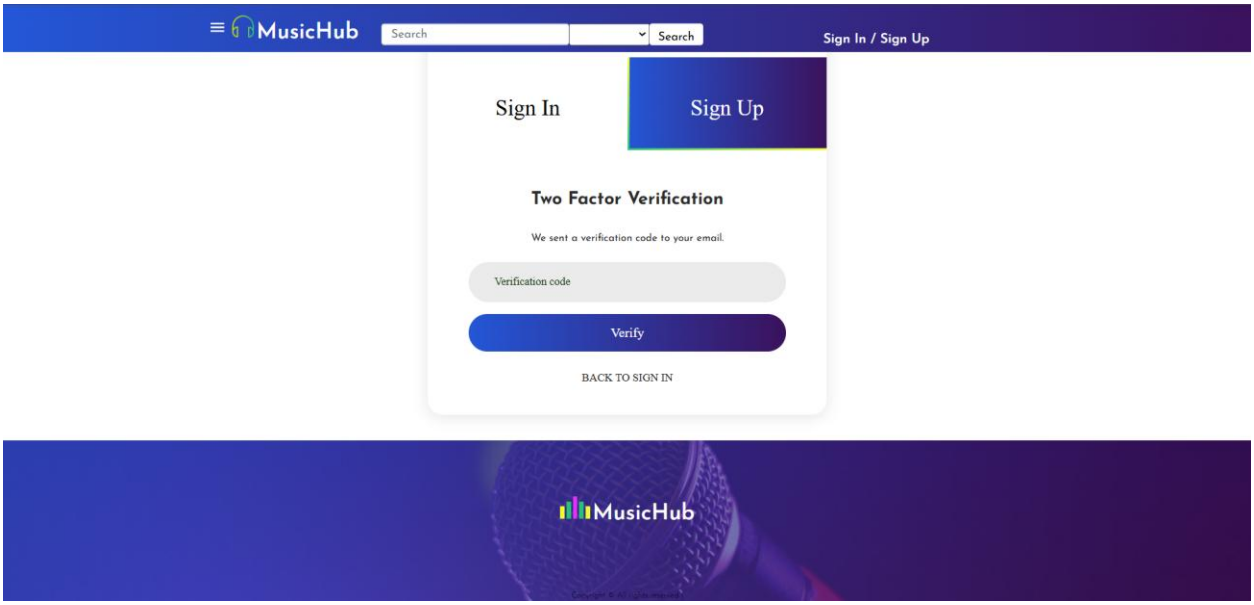


Фиг. 7.6 Форма за вход

В рамките на този процес на регистрирания електронен адрес на потребителя се изпраща еднократен код за потвърждение. Потребителят трябва да въведе получения код в системата, за да завърши успешно процеса на вход и да получи достъп до функционалностите на приложението.



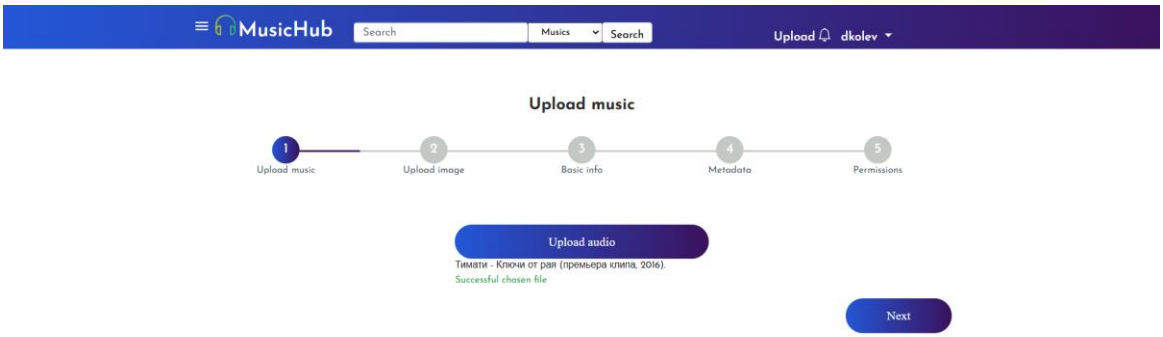
Фиг. 7.7 Two-Factor mail



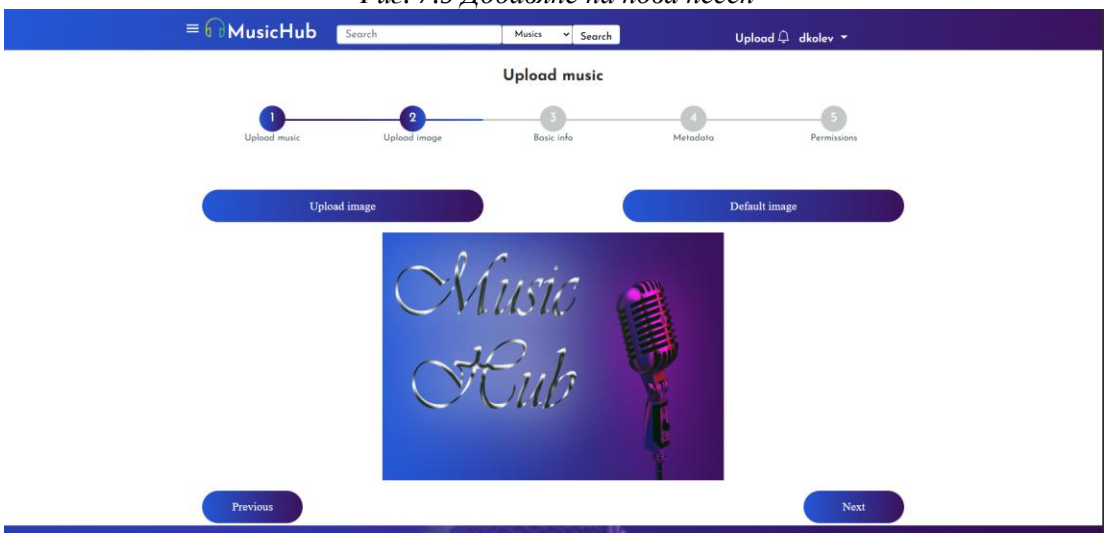
Фиг. 7.8 Въвеждане на код за Two-Factor mail

- **Добавяне на песен**

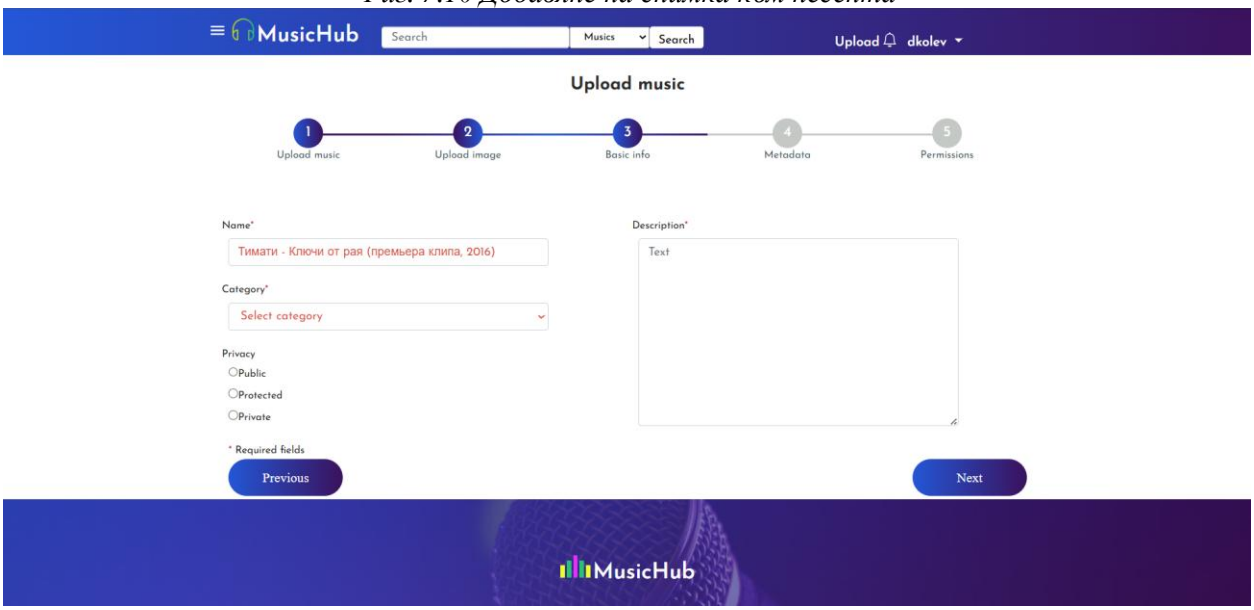
Процесът по добавяне на ново музикално съдържание в платформата е разделен на няколко последователни етапа с цел по-добра организация и валидиране на данните. Стъпките включват: **Upload music**, **Upload image**, **Basic info**, **Metadata** и **Permissions**.



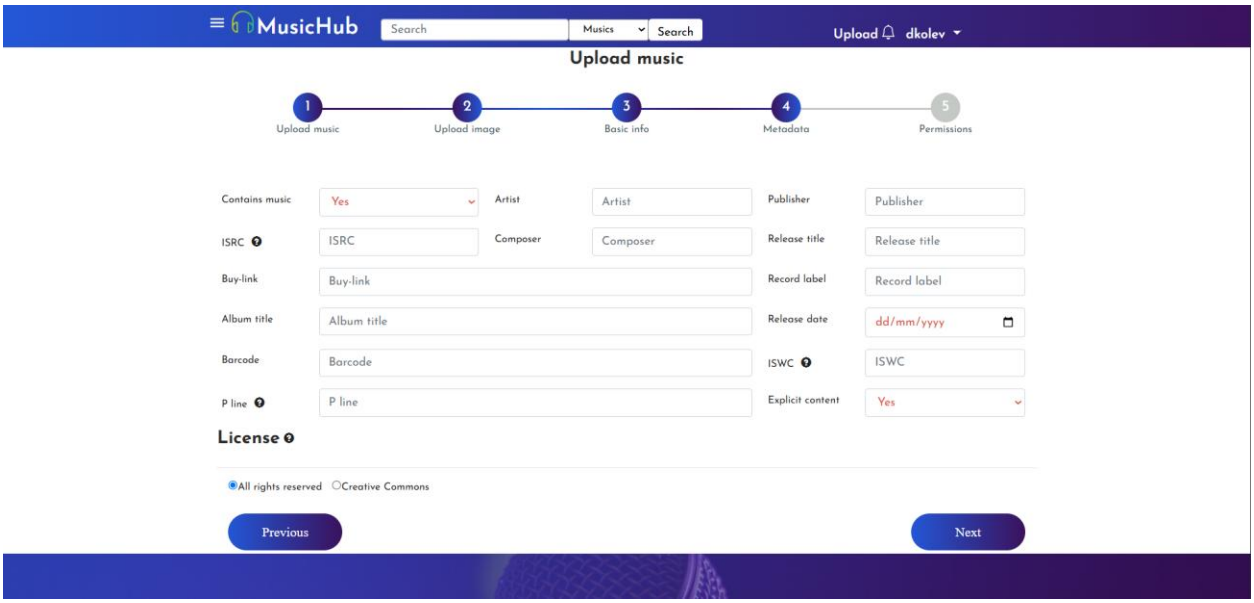
Фиг. 7.9 Добавяне на нова песен



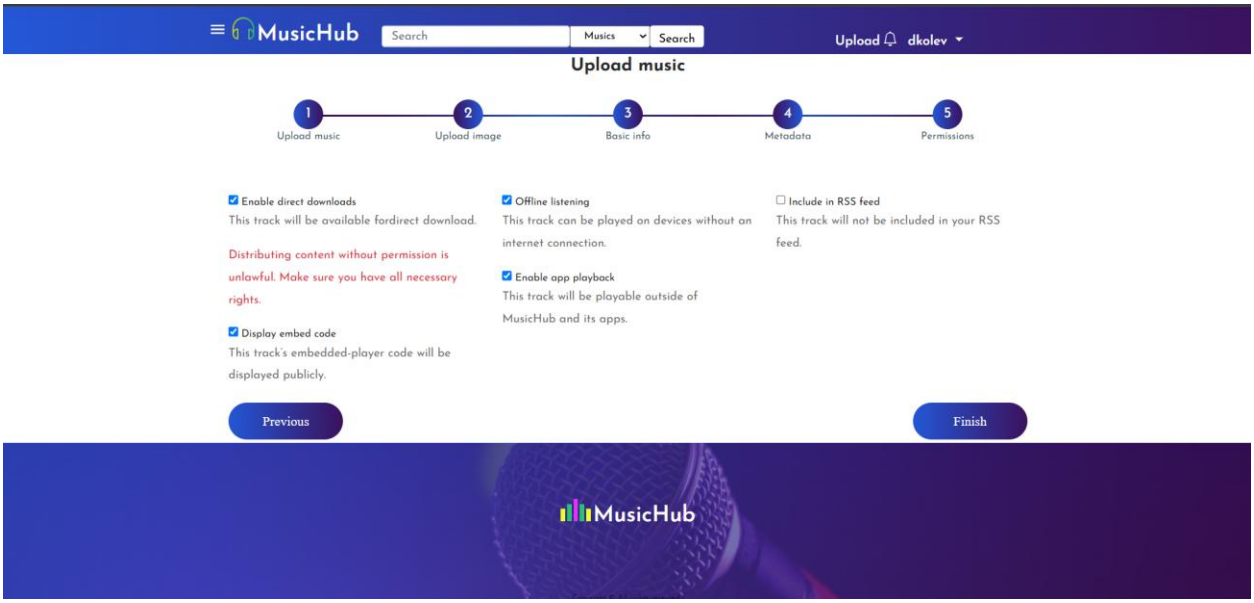
Фиг. 7.10 Добавяне на снимка към песента



Фиг. 7.11 Основна информация към песента



Фиг. 7.12 Metadata



Фиг. 7.12 Permissions

Първоначално потребителят качва музикалния файл, след което се добавя изображение, което служи като визуална корица на песента. В следващата стъпка се попълва основна информация, като заглавие, изпълнител и описание. След това се добавят допълнителни метаданни, които описват технически и съдържателни характеристики на файла. Последният етап включва конфигуриране на права за достъп, чрез които се определя кои потребители могат да слушат съответното музикално съдържание.

В текущата реализация файловете първоначално се съхраняват директно в директория на проекта, което не е оптимално решение от гледна точка на архитектура и скалируемост. Поради тази причина е предвидена бизнес логика за интеграция с външни storage услуги, като **Azure Storage** чрез AzureStorageService и **Mega Storage** чрез

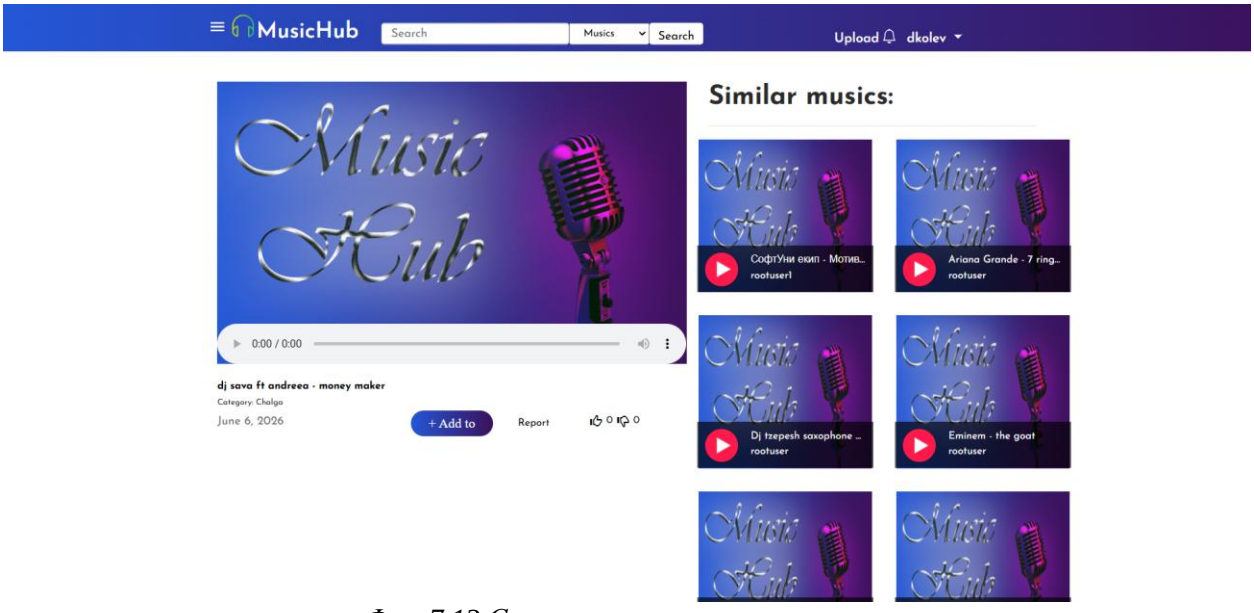
MegaStorageService. Тези услуги осигуряват по-надеждно и централизирано съхранение на файлове.

Важно е да се отбележи, че използването на тези външни storage услуги е свързано с разходи, тъй като те са платени решения. Поради това в рамките на проекта те се разглеждат единствено като архитектурно предвидена възможност, докато текущата имплементация е насочена към учебни и демонстрационни цели, а не към комерсиална експлоатация.

- **Слушане на песен**

Страницата за слушане на музика предоставя възможност на потребителя да възпроизвежда избрано музикално съдържание чрез вграден аудио плейър. Освен основната функционалност за слушане, платформата предлага и система за препоръка на подобни песни. Тези предложения се генерират на база историята на слушане на конкретния потребител и са изцяло базирани на предварително дефинирана бизнес логика, без използване на изкуствен интелект.

На същата страница потребителят разполага с допълнителни интерактивни функции, включително възможност за добавяне на песента към личен плейлист, харесване или нехаресване на съдържанието, както и подаване на сигнал (report) при необходимост. Тези действия позволяват персонализиране на потребителското изживяване и подпомагат управлението на съдържанието в платформата.



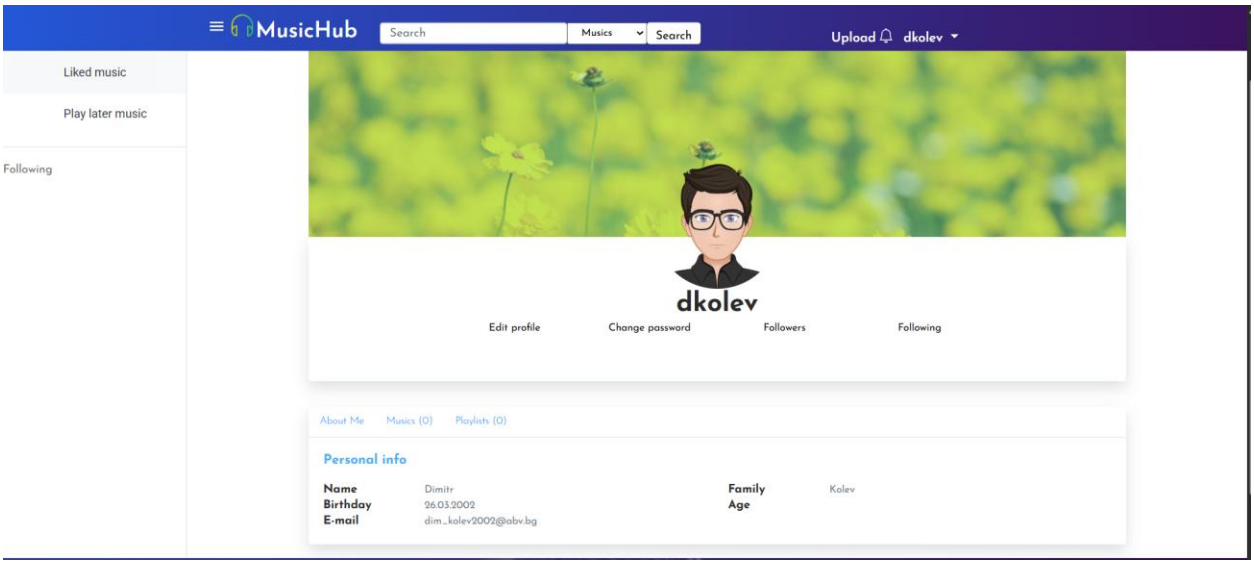
Фиг. 7.13 Страница за слушане на музика

- **Профил на потребителя**

Страницата за потребителски профил предоставя възможност за управление и преглед на личната информация в системата. От тази секция потребителят може да

редактира своите основни данни, както и да извърши промяна на паролата си с цел повишаване на сигурността на акаунта.

Освен настройки на профила, страницата включва и социални функционалности. Потребителят може да преглежда своите последователи, както и списък с профилите, които следва. По този начин се осигурява възможност за социално взаимодействие между потребителите в платформата и по-лесно проследяване на активността в мрежата.

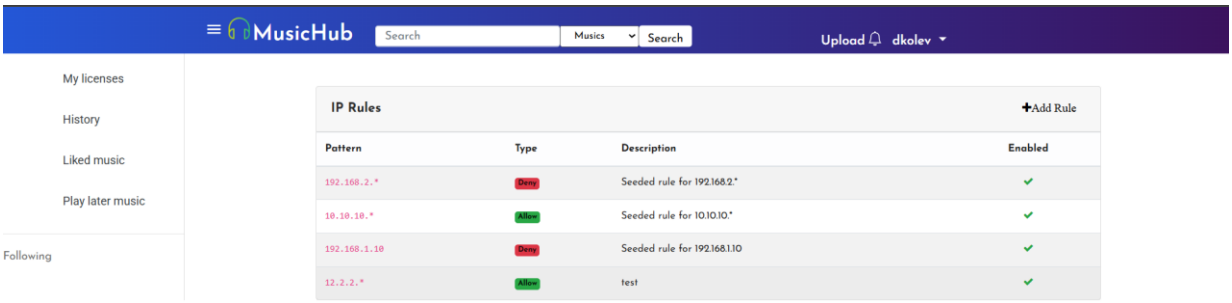


Фиг. 7.13 Страница за слушане на музика

- **Управление на IP конфигурация**

Страницата за IP конфигурация предоставя възможност за ограничаване на достъпа до профила или системата въз основа на IP адреси. Чрез нея потребителят или администраторът може да дефинира списък с разрешени IP адреси, от които е позволено влизане в приложението, както и списък със забранени IP адреси, от които достъпът се блокира.

Тази функционалност се използва като допълнителен слой за сигурност, като позволява по-прецизен контрол върху мрежовия достъп до системата. По този начин се намалява рискът от неоторизиран достъп и се повишава общата защита на платформата.



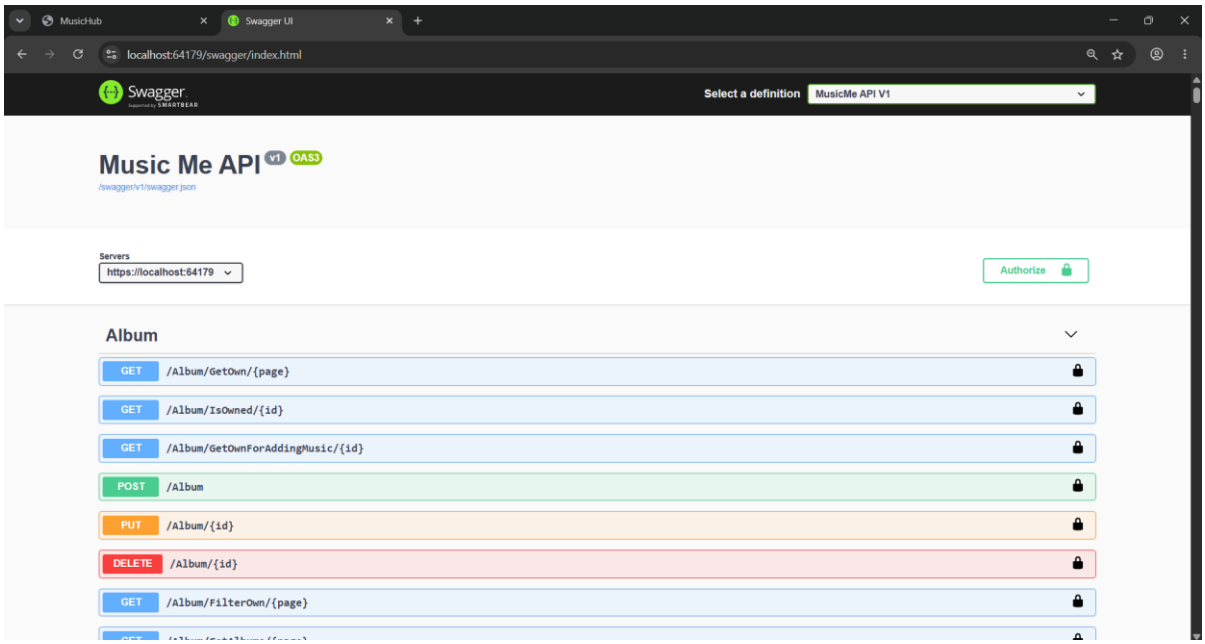
Фиг. 7.13 Страница Управление на IP конфигурация

- **Стартиране на swagger**

За тестване и документация на Web API в проекта е използван Swagger (OpenAPI). След стартиране на приложението Swagger интерфейсът е достъпен на адрес от вида:

https://localhost:64179/swagger/index.html

или на съответния порт, на който е стартирано приложението, последван от /swagger/index.html.

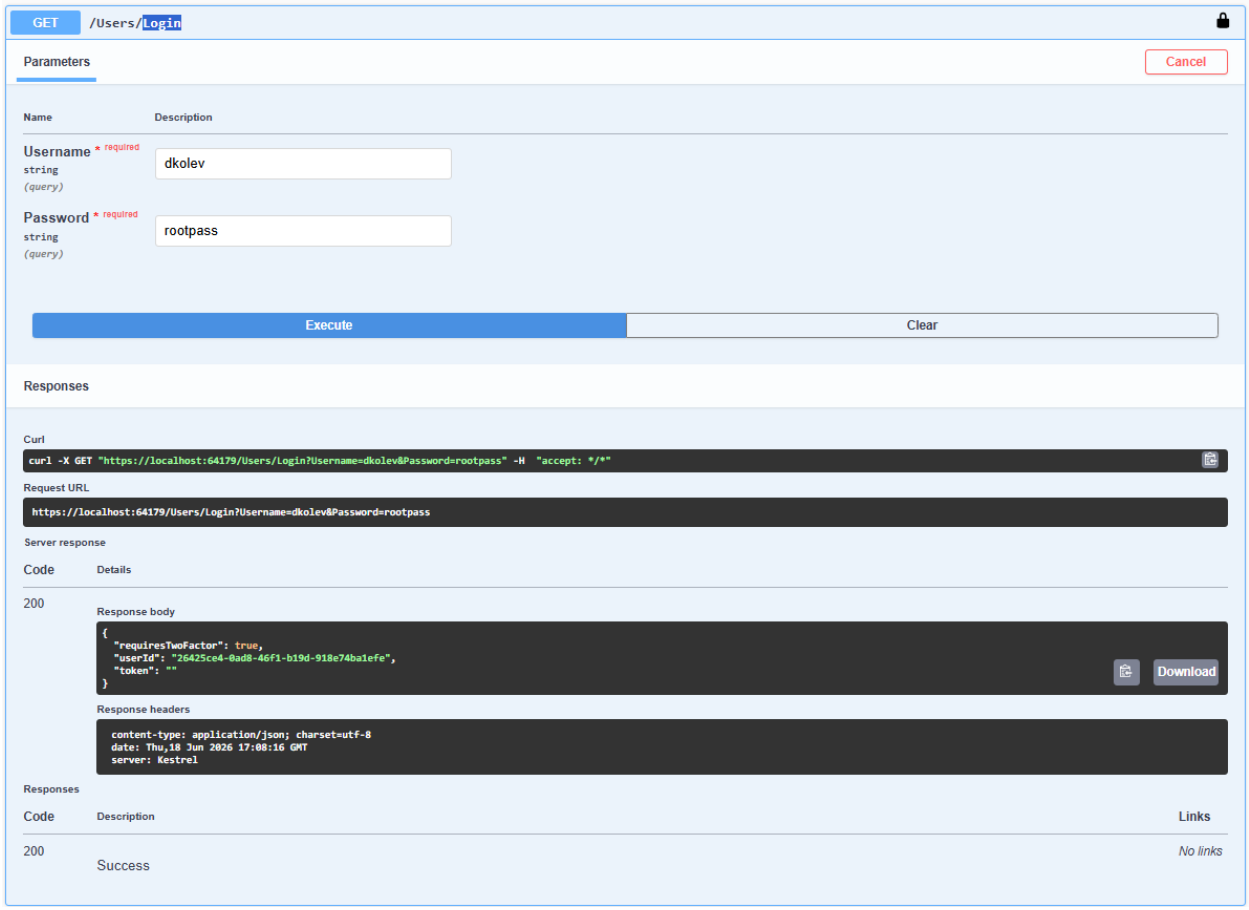


Фиг. 7.15 Swagger

В интерфейса на Swagger са визуализирани всички налични контролери и техните действия (Actions), като всяка операция може да бъде изпълнена директно през браузъра.

За удостоверяване на потребител първоначално се извиква методът POST **/Users/Login**, като се попълват потребителско име и парола. При успешно изпълнение методът връща JWT токен или изисква двуфакторна автентикация чрез електронна поща. В случай че е активирана двуфакторна автентикация, отговорът е подобен на следния:

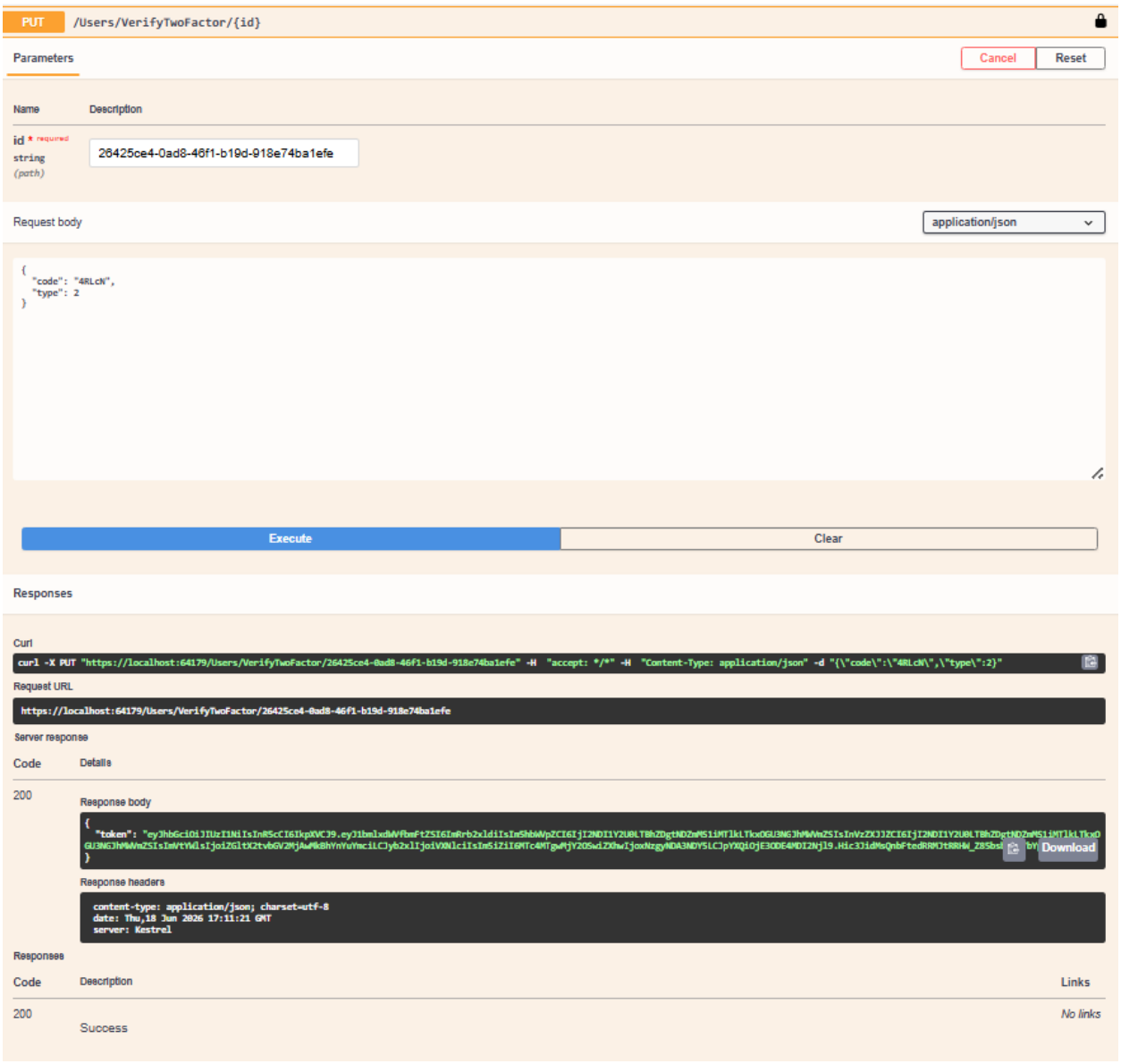
```
{
  "requiresTwoFactor": true,
  "userId": "26425ce4-0ad8-46f1-b19d-918e74ba1efe",
  "token": ""
}
```



Фиг. 7.16 Users/Login отговор

Когато е необходима двуфакторна автентикация, трябва да бъде извикан методът POST /Users/VerifyTwoFactor/{id}, където параметърът id представлява идентификатора на потребителя (userId), получен от предходната заявка. В тялото на заявката се изпращат полученият код за потвърждение и типът на двуфакторната автентикация:

```
{
  "code": "code",
  "type": 2
}
```

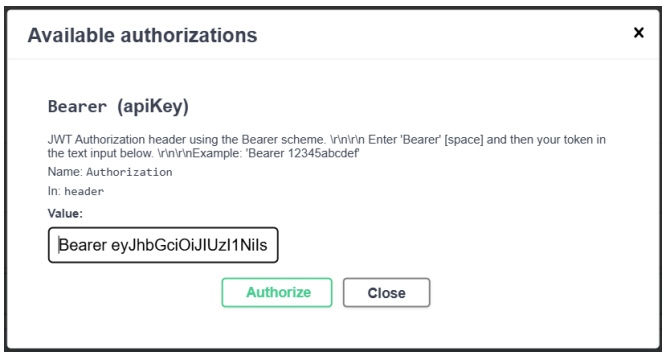


Фиг. 7.16 /Users/VerifyTwoFactor/{id} отговор

След успешно потвърждение методът връща JWT токен, който се използва за достъп до защитените крайни точки на приложението.

За използване на получения токен в Swagger се избира бутонът Authorize, след което в отворения прозорец се въвежда стойността в следния формат:

Bearer <JWT_TOKEN>



Фиг. 7.17 Логване чрез bearer token

След успешна авторизация Swagger автоматично добавя токена към всички последващи заявки към защитените API методи, което позволява тяхното тестване без необходимост от повторно удостоверяване.

ЗАКЛЮЧЕНИЕ

В рамките на настоящата дипломна работа беше проектирана и разработена уеб базирана платформа MusicHub, предназначена за публикуване, управление и възпроизвеждане на музикално съдържание. В процеса на разработка бяха анализирани съществуващи решения, дефинирани бяха функционалните и нефункционалните изисквания към системата, избрани бяха подходящи технологии и беше реализирана цялостна архитектура, разделена на клиентска и сървърна част.

Разработеното приложение предоставя всички основни функционалности, поставени като цели в началото на проекта – регистрация и удостоверяване на потребители, управление на потребителски профили, публикуване и управление на музикални произведения, създаване на плейлисти, търсене на съдържание, взаимодействие между потребителите чрез харесвания и коментари, както и административен панел за управление на системата. Освен това са реализирани механизми за защита на приложението, включително удостоверяване и авторизация чрез JWT, защита срещу CSRF атаки, ограничаване на достъпа чрез роли, IP филтриране и други добри практики за сигурност.

Полученото приложение може да бъде използвано като напълно функционираща уеб платформа и успешно удовлетворява поставените функционални изисквания. Интерфейсът е интуитивен и удобен за работа, а използваните технологии осигуряват добра производителност, лесна поддръжка и възможност за бъдещо разширяване. Реализираната архитектура позволява добавяне на нови функционалности без съществени промени в съществуващия код.

Въпреки че платформата не може да се конкурира пряко с утвърдени международни решения като Spotify, SoundCloud или YouTube Music, зад които стоят големи екипи от разработчици и значителни финансови ресурси, тя успешно демонстрира реализацията на съвременна система за управление и разпространение на музикално съдържание. Проектът изпълнява поставените цели и показва практически умения за проектиране, разработване и защита на уеб приложения, като същевременно представлява добра основа за надграждане до продукт с по-широко приложение.

Като бъдещо развитие на системата могат да бъдат реализирани функционалности като интеграция с външни музикални услуги, препоръчваща система, базирана на машинно обучение, мобилно приложение за Android и iOS, известия в реално време, офлайн режим за възпроизвеждане на музика, както и разширени аналитични инструменти за изпълнителите. Реализирането на подобни възможности би повишило конкурентоспособността на платформата и би подобрило потребителското изживяване.

В заключение може да се приеме, че поставените цели и задачи на дипломната работа са успешно изпълнени. Разработената система представлява завършено, надеждно и разширяемо софтуерно решение, което отговаря на съвременните изисквания за уеб приложения и демонстрира ефективното прилагане на използваните технологии и добрите практики при разработката на софтуер.

ЛИТЕРАТУРА

Изброени и описани ползваните източници, подредбата е по азбучен ред на автора

[1] Softuni Курс - https://softuni.bg/trainings/2419/asp-net-core-mvc-june-2019#lesson-12350

[2] Microsoft документация - https://dotnet.microsoft.com/en-us/apps/aspnet

[3] Entity framework документация - https://learn.microsoft.com/en-us/ef/

[4] xUnit - https://xunit.net/?tabs=cs

[5] MyTested.AspNetCore.Mvc примери- https://www.youtube.com/watch?v=cHe4tCDdKAA&list=PLLGdqRi7N09YoomAKadOjLopq0sIr_PKT

[5] Angular документация - https://angular.dev/

ПРИЛОЖЕНИЯ

GitHub - https://github.com/dimitarkole/MusicHub

Azure DEVOPS - https://dev.azure.com/dimkolev2002/MusicHub%202.0